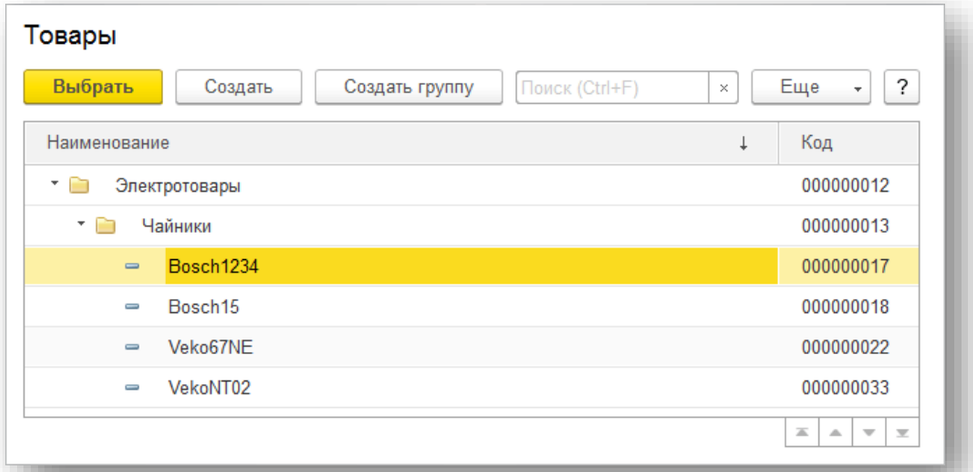
Современные сервисные платформенные решения.

Лекция 2. Прикладные объекты конфигурации. Константы, перечисления и справочники

Старший преподаватель кафедры «Информационные системы»
Шеркунов Вячеслав Владимирович



Ульяновский государственный технический университет

2025

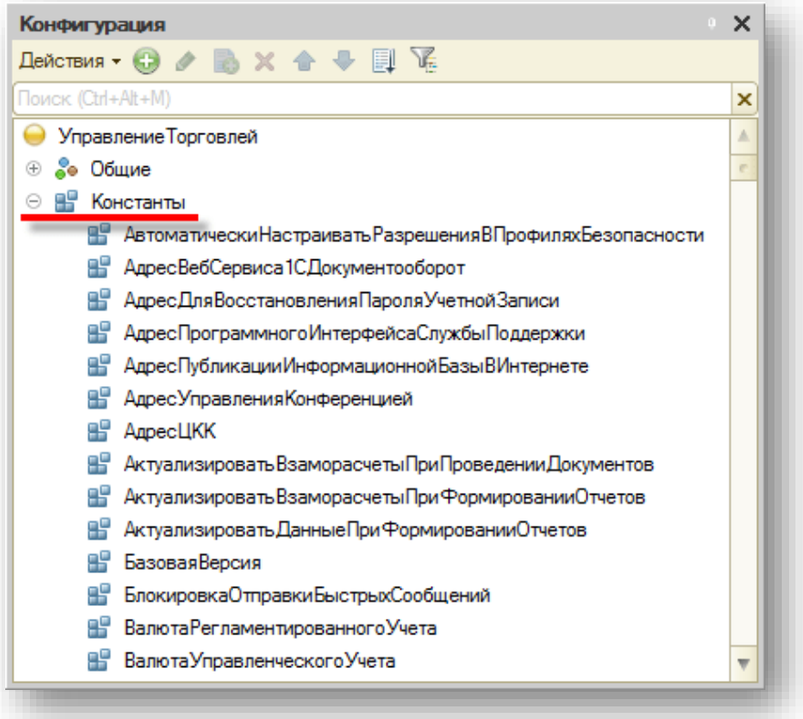
Содержание лекции

- Константы
- Перечисления
  1. Общие сведения
  2. Создание
  3. Значение перечисления
  4. Перечисление в реквизите объекта
  5. Перечисление на управляемой форме
  6. Программная работа
- Справочники:
  1. Общие сведения
  2. Структура
  3. Формы
  4. Иерархические справочники
  5. Подчиненные справочники
  6. Программная работа со справочниками
  7. Ошибки

Константы. Общие сведения.

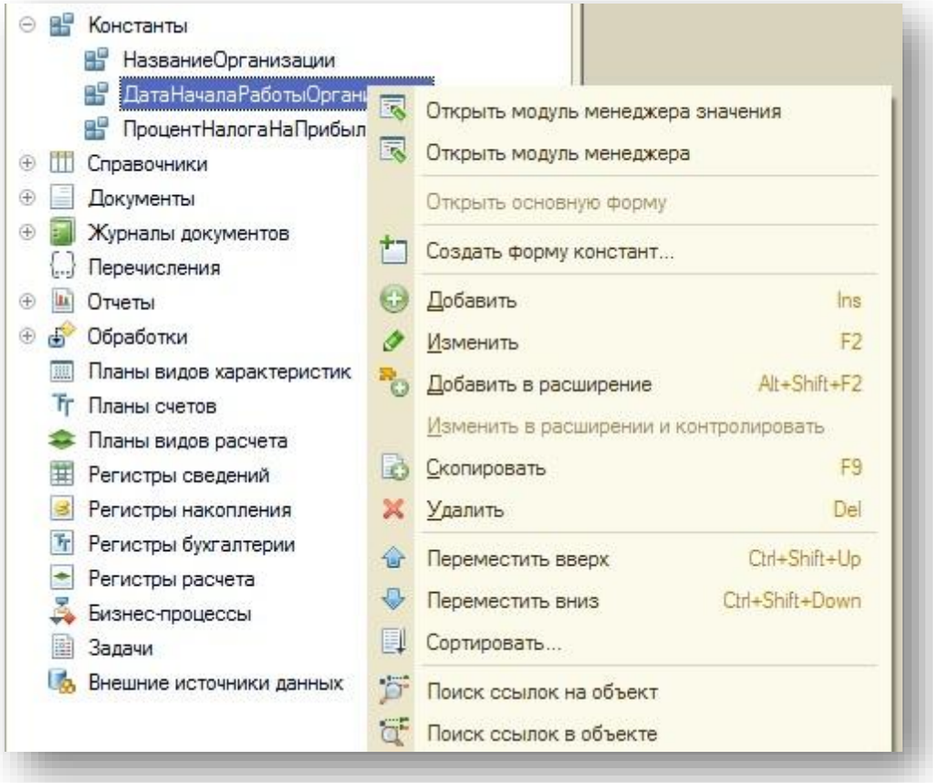
Константы – это прикладные объекты конфигурации. Они позволяют хранить в информационной базе данные, которые не изменяются во времени, или изменяются очень редко. Каждая константа позволяет хранить одно значение.

Например, в константе может храниться наименование предприятия, его ИНН и другая информация. В прикладном решении может быть создано произвольное количество констант:



Константы. Общие сведения.

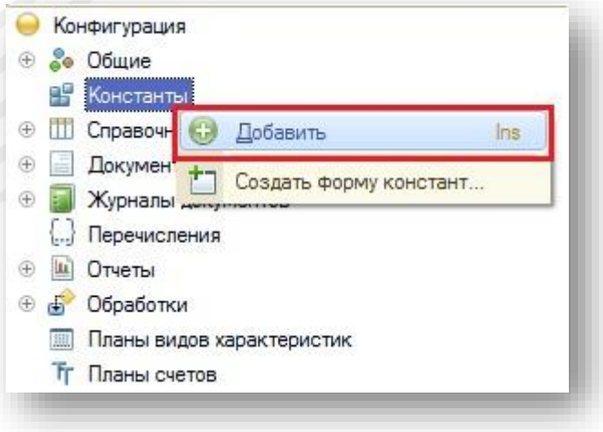
Смысл констант в языке программирования 1С несколько другой, чем в других языках. В 1С константы хранят постоянную или условно постоянную информацию, которая редко изменяется. Если в других языках программирования пользователь не может менять константу, то в платформе 1С имеется такая возможность, но по определению делать он это должен крайне редко.

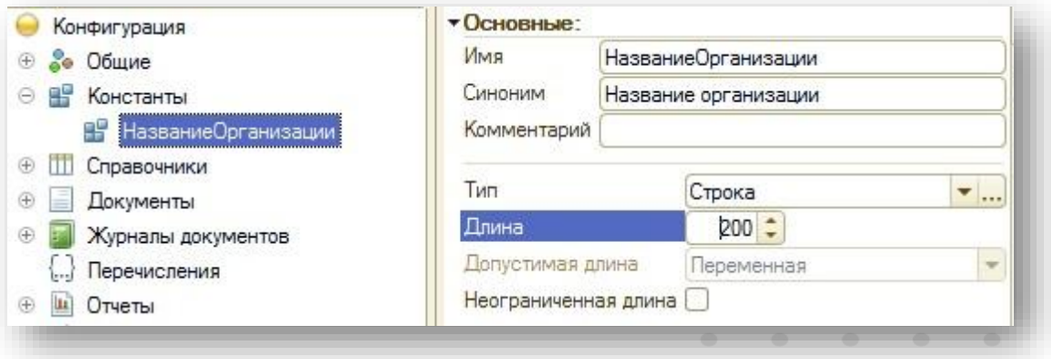


Константы. Создание константы.

Нужно выбрать «*Константы*» в дереве метаданных конфигуратора 1С, выделить её, и вызвать правой кнопкой мышки контекстное меню, и выполнить команду «Добавить».

После выполнения этой команды, константа будет создана, а в правой части рабочего стола откроется палитра свойств новой константы. Здесь можно указать название константы и её тип. Пусть первая константа будет называться Название Организации(тип строка (200)).

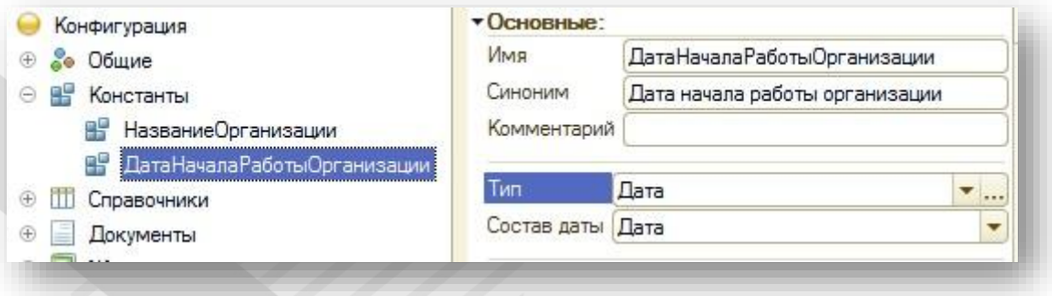




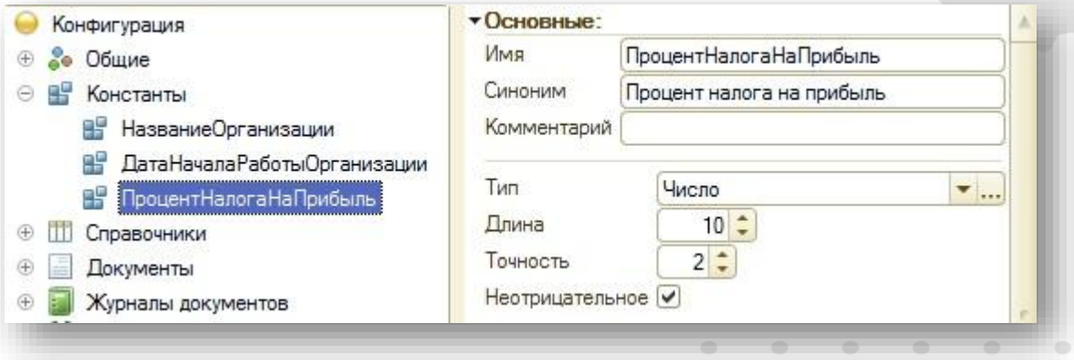
Константы. Создание константы.

Создание констант с примитивными типами:

Дата начала работы организации (тип Дата)

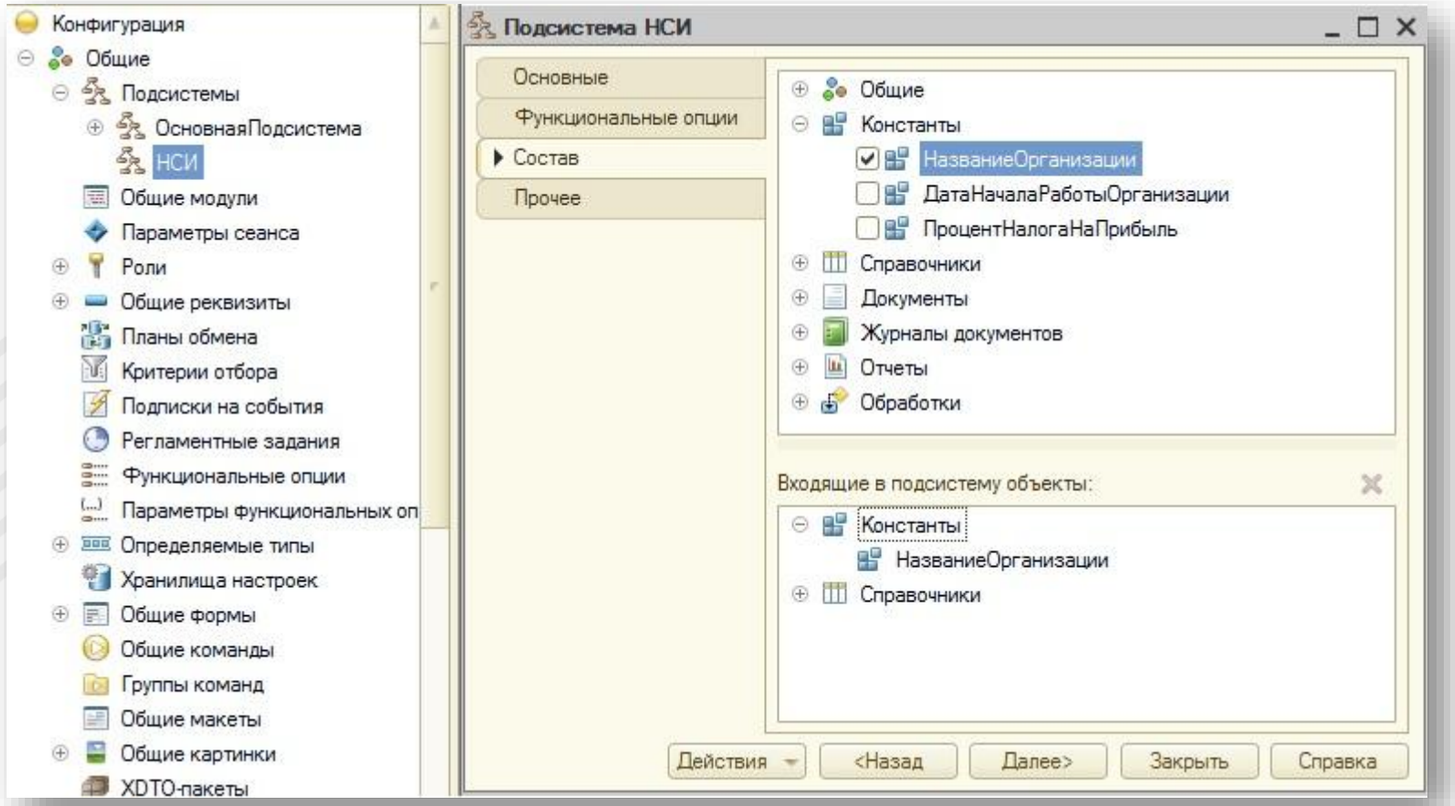


Процент налога на прибыль (тип число)



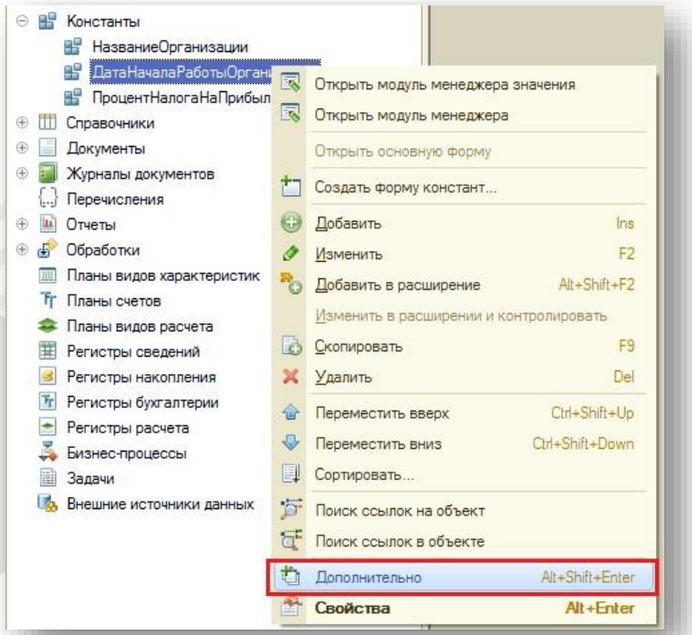
Константы. Константы в командном интерфейсе.

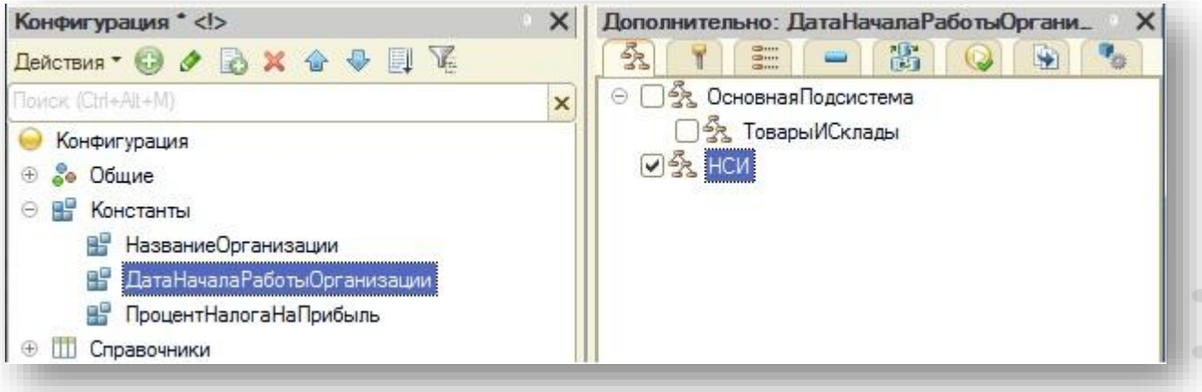
Первый способ добавления в командный интерфейс.
Открыть редактор нужной подсистемы, и на закладке «*Состав*» включить вхождение нужной константы в подсистему.



Константы. Константы в командном интерфейсе.

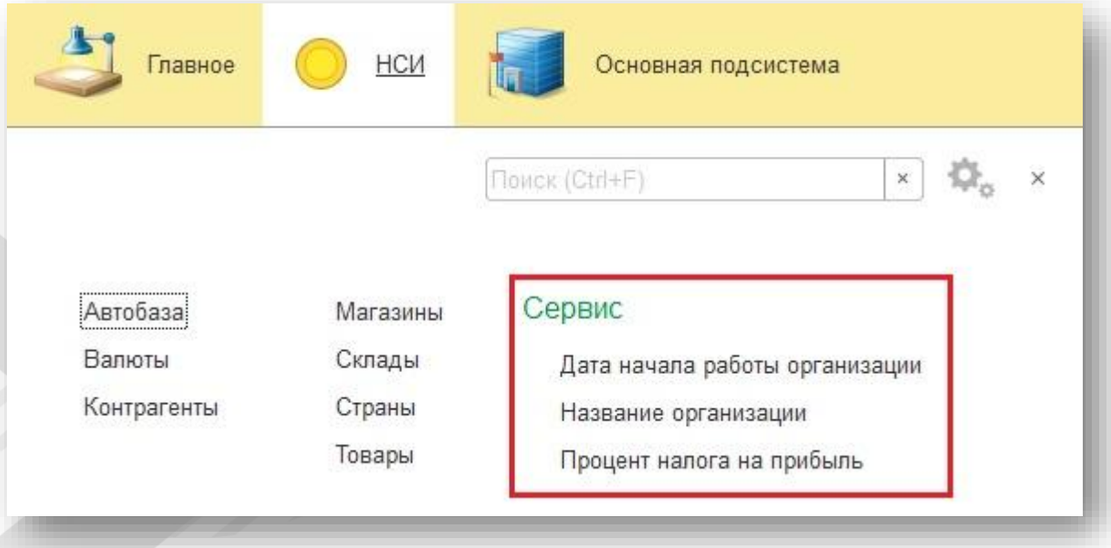
Второй способ добавления в командный интерфейс.
У константы вызвать контекстное меню, в котором выполнить команду «*Дополнительно*». В открывшейся в правой части окне «*Дополнительно*», на закладке «*Подсистемы*» нужно установить флаг рядом с той подсистемой, в которую хотите включить константу.

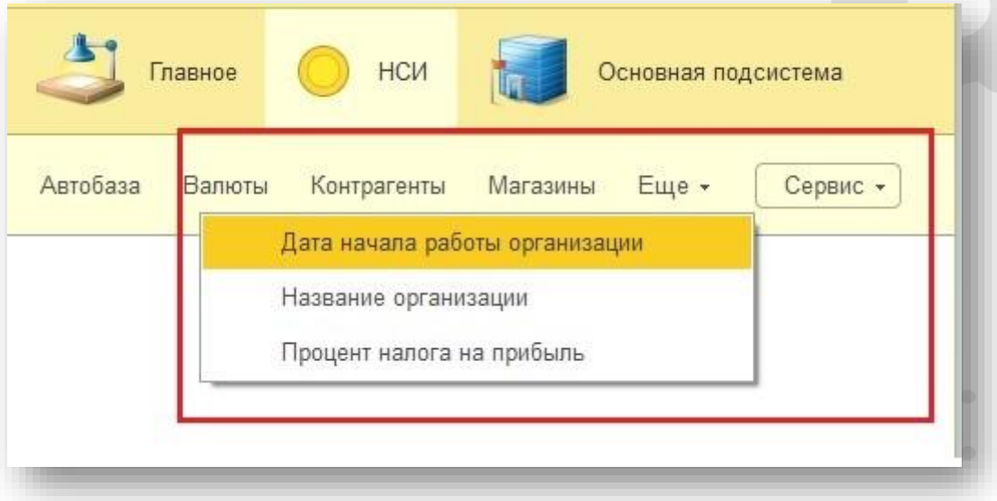




Константы. Константы в командном интерфейсе.

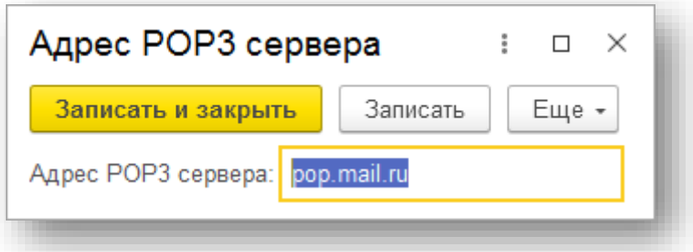
После включения всех констант в подсистему и обновления базы данных, все новые константы можно найти в панели действий «*Сервис*».



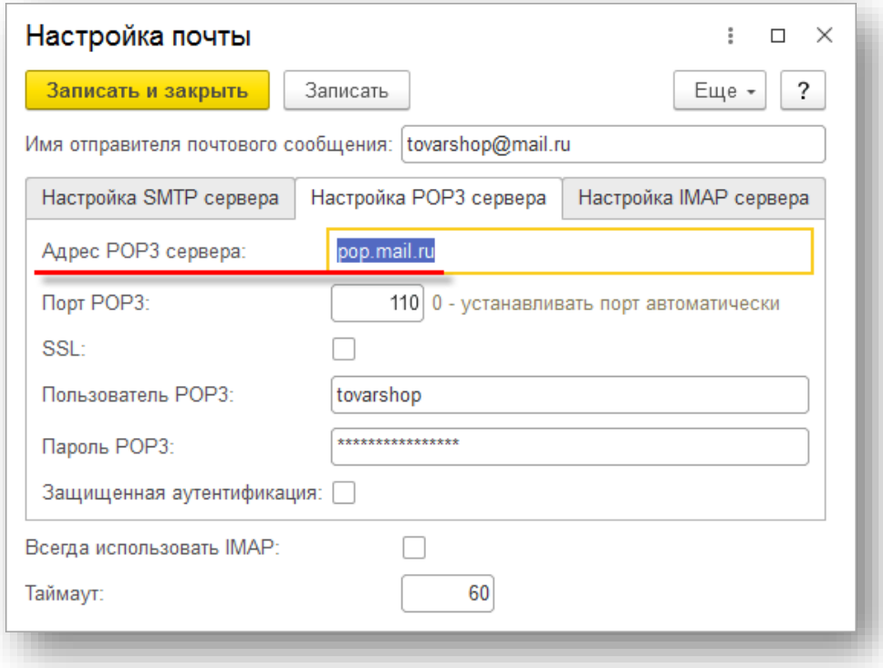


Константы. Форма констант.

Для того, чтобы пользователь мог просматривать и изменять значения констант, система может автоматически сгенерировать форму константы:



Наряду с этим разработчик имеет возможность создать собственную форму, которую система будет использовать вместо формы по умолчанию:

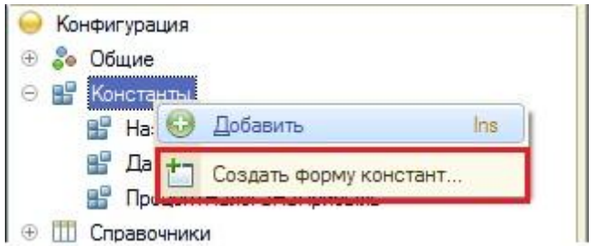


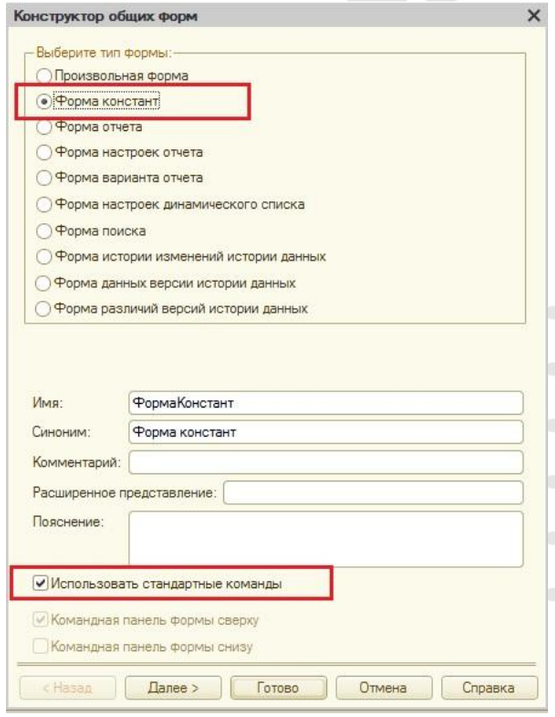
Константы. Форма констант.

Редактировать константы в каждой отдельной форме иногда не совсем удобно, гораздо практичнее, чтобы все константы можно было отредактировать в одной форме. Для этих целей можно создать форму констант, посредством которой и будут редактироваться все константы, которые разработчик включит в эту форму.

Для этого нужно выделить ветку метаданных *Константы*, вызвать контекстное меню, и выполнить команду «Создать форму констант...» этого меню. Откроется конструктор общих форм.

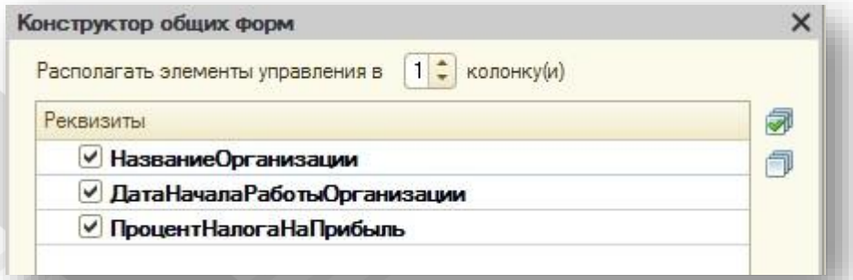
Стоит обратить внимание на *Тип Формы*, и на свойство *Использовать стандартные команды*. Первое указывает, что это форма констант, а при установке второго будут созданы стандартные команды на открытие этой формы.

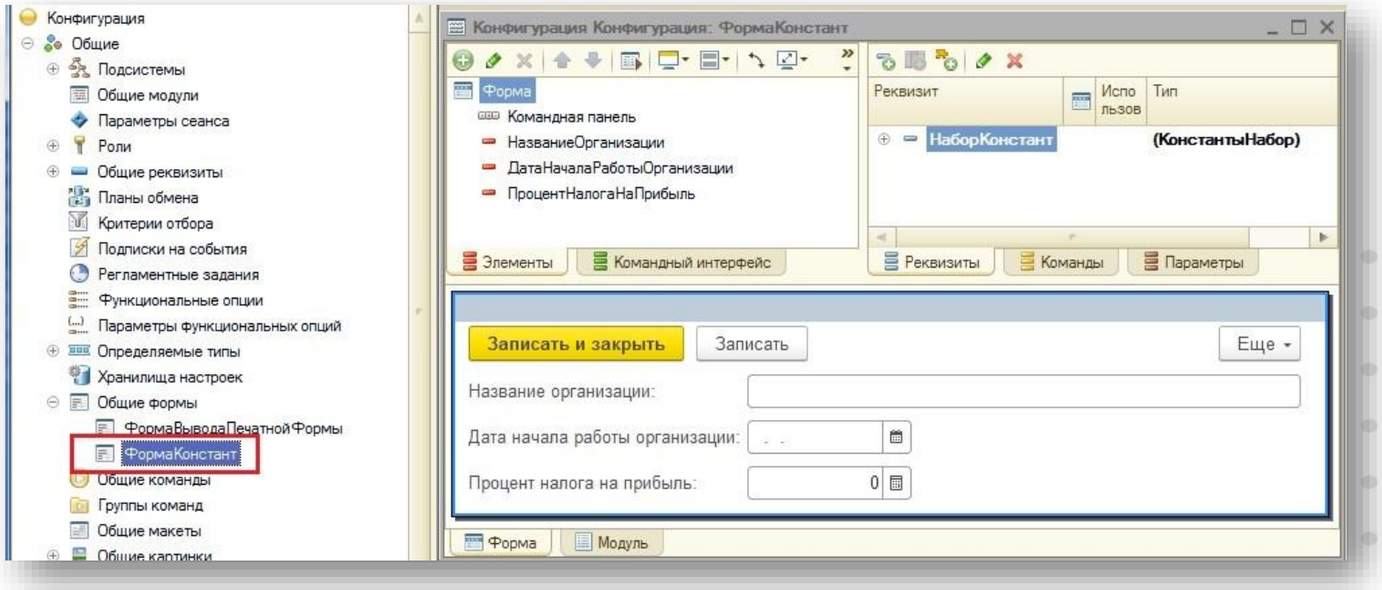




Константы. Форма констант.

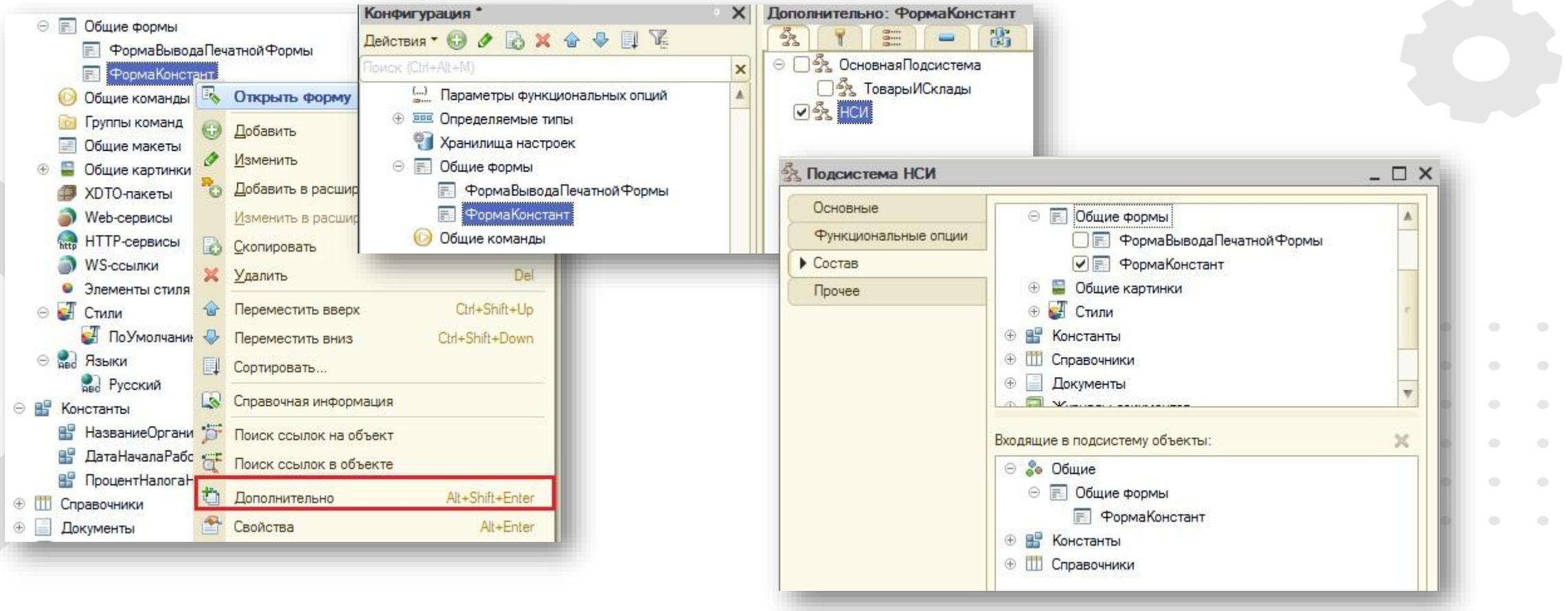
После нажатия кнопки «*Далее*», необходимо будет выбрать, какие константы войдут в эту форму. После того, как выбрали нужные константы, нажимаем кнопку «*Готово*», и управляемая форма констант будет создана. Форма добавилась в ветку «*Общие формы*» ветки «*Общие*» дерева метаданных 1С.





Константы. Форма констант.

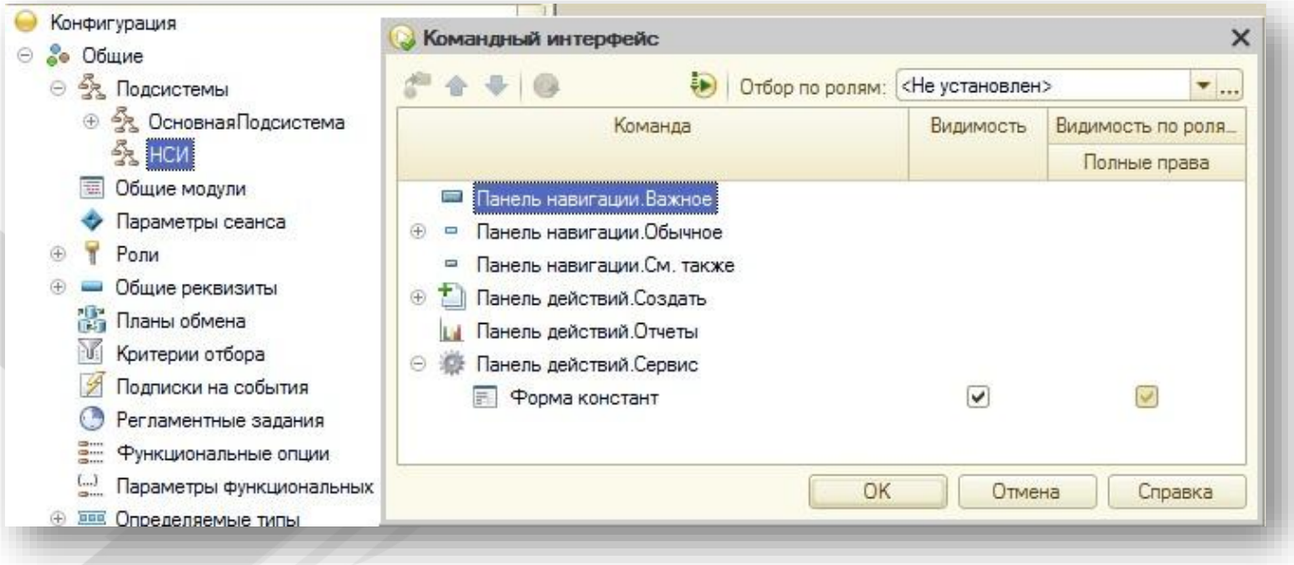
Для включения общей формы констант в подсистему необходимо вызвать контекстное меню общей формы констант и выполнить команду «Дополнительно».
В открывшемся окне нужно включить форму констант в подсистему 1С. Также эту форму можно добавить в подсистему, напрямую во время редактирования подсистемы.



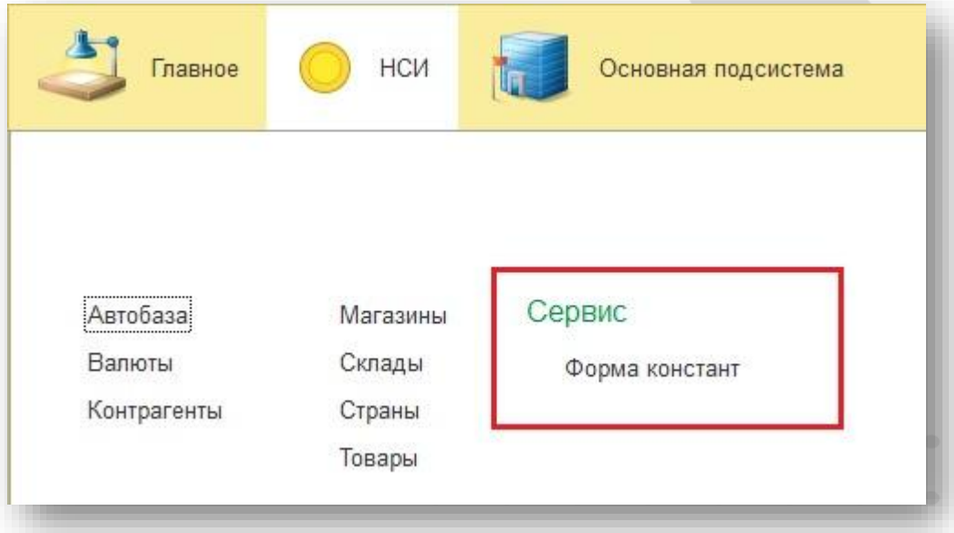
Константы. Форма констант.

У всех остальных констант можно отключить свойство «*Использовать стандартные команды*», чтобы в командном интерфейсе отображалась только новая форма констант.

Как в командном интерфейсе подсистемы в конфигураторе.

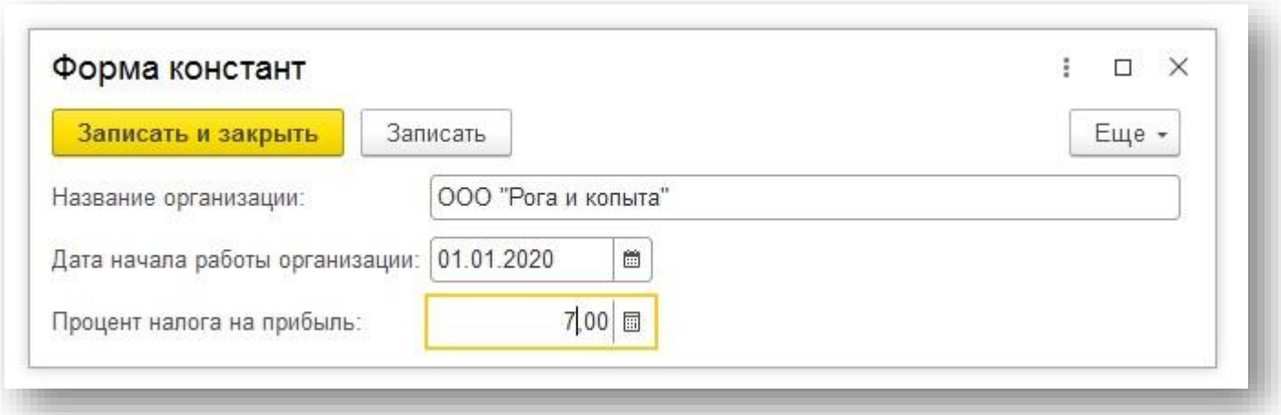


Так и в пользовательском режиме



Константы. Форма констант.

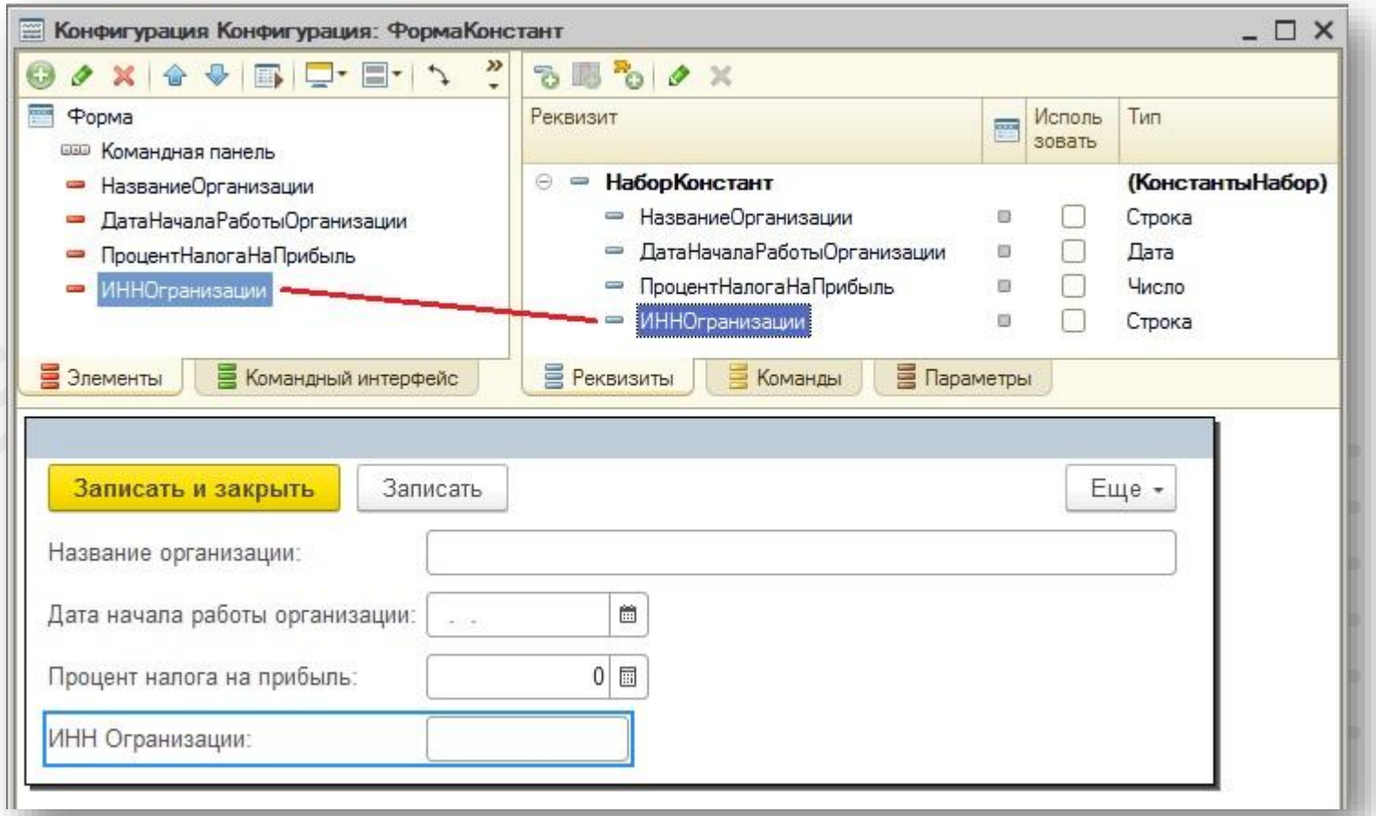
Если эту форму открыть, то в ней будут поля ввода для всех констант, которые были добавлены при создании этой формы.



Константы. Форма констант.

Что делать, если форма уже создана, но позже добавили новую константу, например «ИНН».

Тогда следует добавить новую константу на форму. Для этого, нужно открыть форму на редактирование, развернуть в закладке «*Реквизиты*» дерево «*НаборКонстант*», найти в этом дереве новую константу и перетащить её в элементы формы.



Константы. Программная работа с константами.

КАК ПОЛУЧИТЬ КОНСТАНТУ 1С

Для того чтобы получить значение какой-либо константы, нужно обратиться к менеджеру конкретной константы посредством менеджера всех констант, а потом воспользоваться методом Получить.

Например, для получения и вывода значения константы можно использовать следующий код:

```
&НаСервереБезКонтекста
Процедура ПолучитьНаСервере()
    НазваниеОрганизации = Константы.НазваниеОрганизации.Получить();
    Сообщить(НазваниеОрганизации);
КонецПроцедуры

&НаКлиенте
Процедура Получить(Команда)
    ПолучитьНаСервере();
КонецПроцедур
```

Константы. Программная работа с константами.

УСТАНОВИТЬ ЗНАЧЕНИЕ КОНСТАНТЫ

Если нужно программно установить значение новой константы, или изменить значение уже существующей константы, то можно воспользоваться тем же менеджером константы и его методом *Установить*.

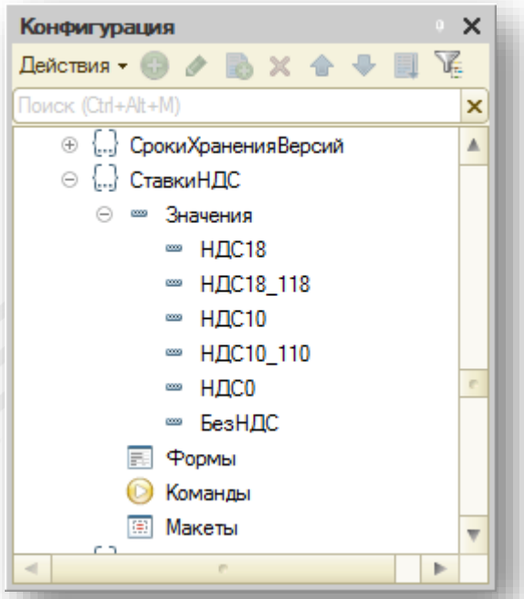
```
&НаСервереБезКонтекста
Процедура УстановитьНаСервере()
    Константы.ПроцентНалогаНаПрибыль.Установить(25);
КонецПроцедуры

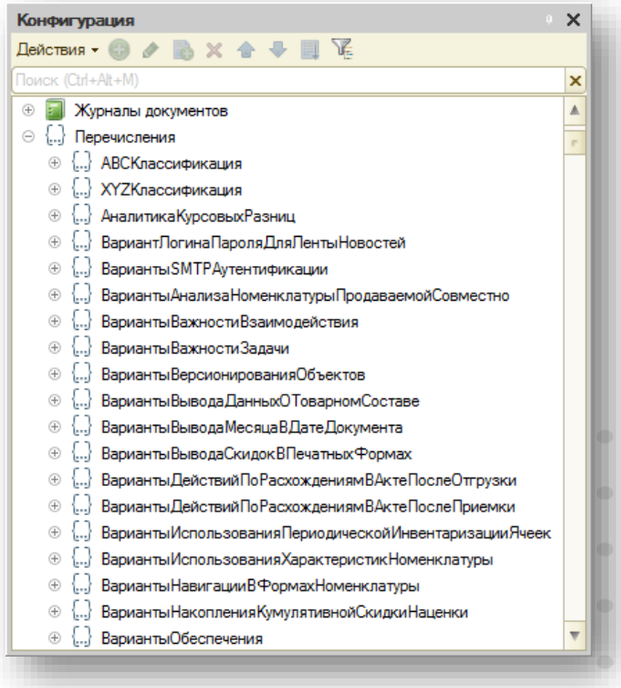
&НаКлиенте
Процедура Установить(Команда)
    УстановитьНаСервере();
КонецПроцедуры
```

В параметрах метода менеджера *Установить* необходимо указать новое значение константы. Тип значения в параметре должен совпадать с типом константы. Точно также как и с получением значения, устанавливать значение константы нужно в серверном контексте.

Перечисления. Общие сведения.

Перечисления — это прикладные объекты конфигурации. Они позволяют хранить в информационной базе наборы значений, которые не изменяются в процессе работы прикладного решения. Например, это может быть перечисление возможных ставок НДС (БезНДС, НДС18 и т. д.), перечисление состояния заказов (Запланировано, ВРаботе, Выполнено) и пр.:



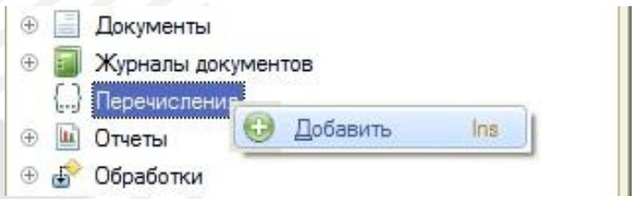


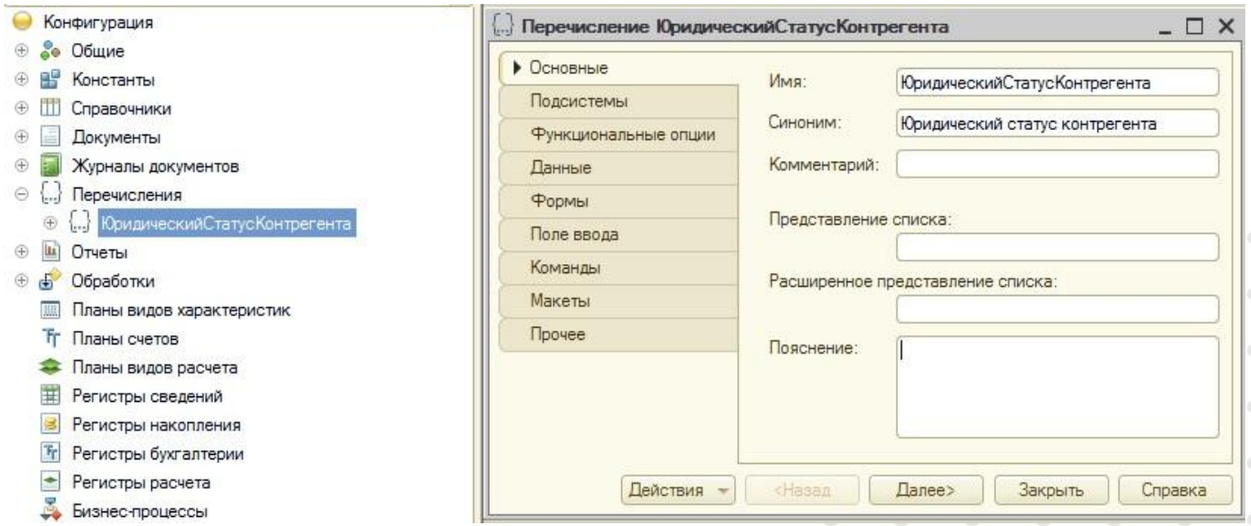
В прикладном решении может быть создано произвольное количество перечислений.

Перечисления. Создание перечисления.

Для создания перечислений необходимо в конфигураторе 1С, в дереве метаданных, найти ветку перечисления, выделить её, вызвать контекстное меню и воспользоваться единственной командой этого меню «*Добавить*».

После этих действий будет открыт конструктор редактирования перечислений, где на закладке «*Основные*», можно ввести имя перечисления и синоним.

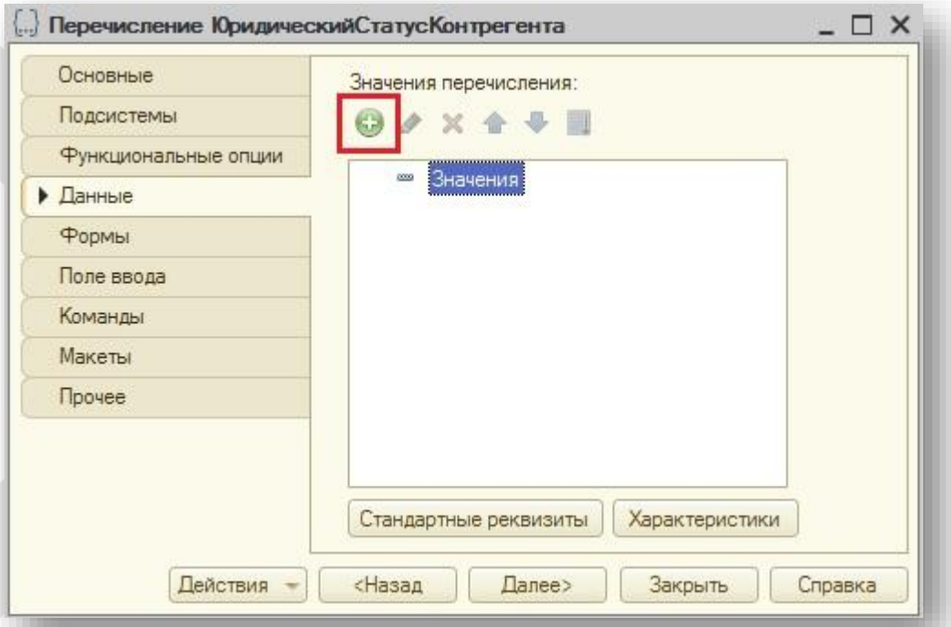


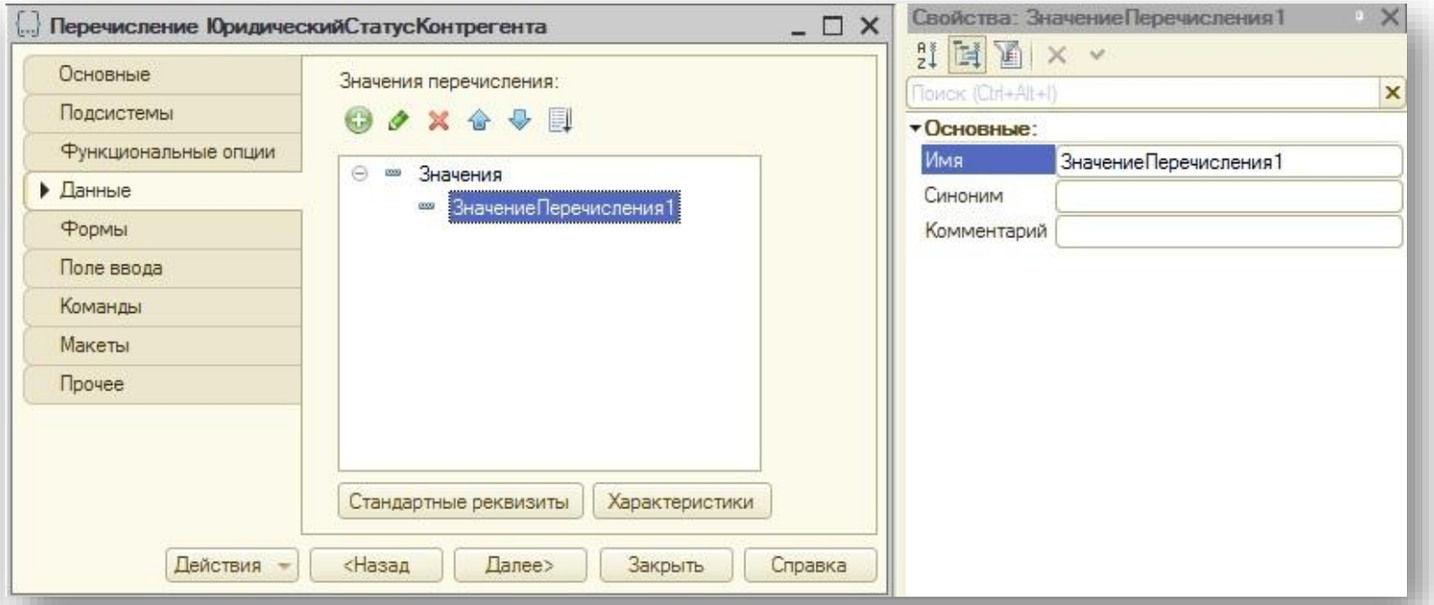


Перечисления. Значение перечисления.

Перечисление создано, но само по себе перечисление не имеет ни какого смысла, в нем должны быть различные значения. Нужно их добавить. Для этого перейдем на закладку «*Данные*», где имеется список значений перечислений. Первоначально он пустой, и нужно добавить в него несколько значений, для этого необходимо выполнить команду «*Добавить*» в меню списка значений.

После выполнения команды, значение будет создано, и справа откроется палитра свойств этого значения. В неё вы можете задать имя конкретного значения, синоним и комментарий.

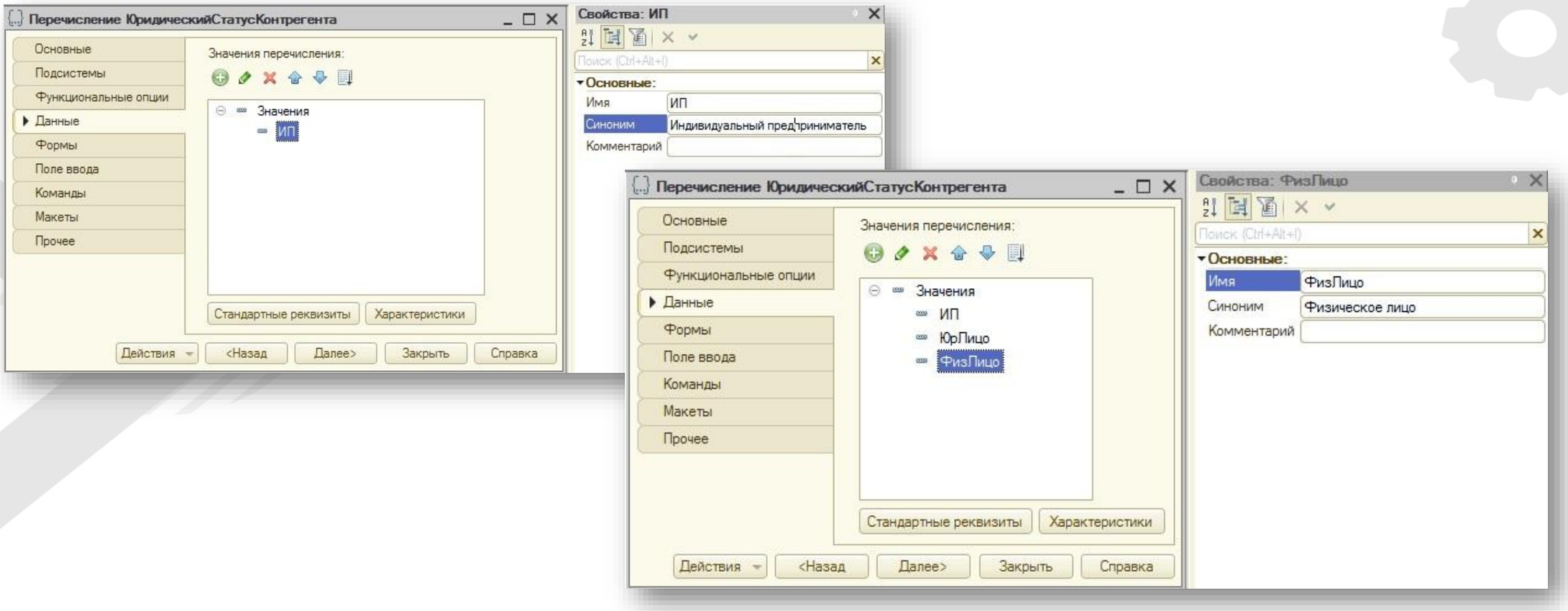




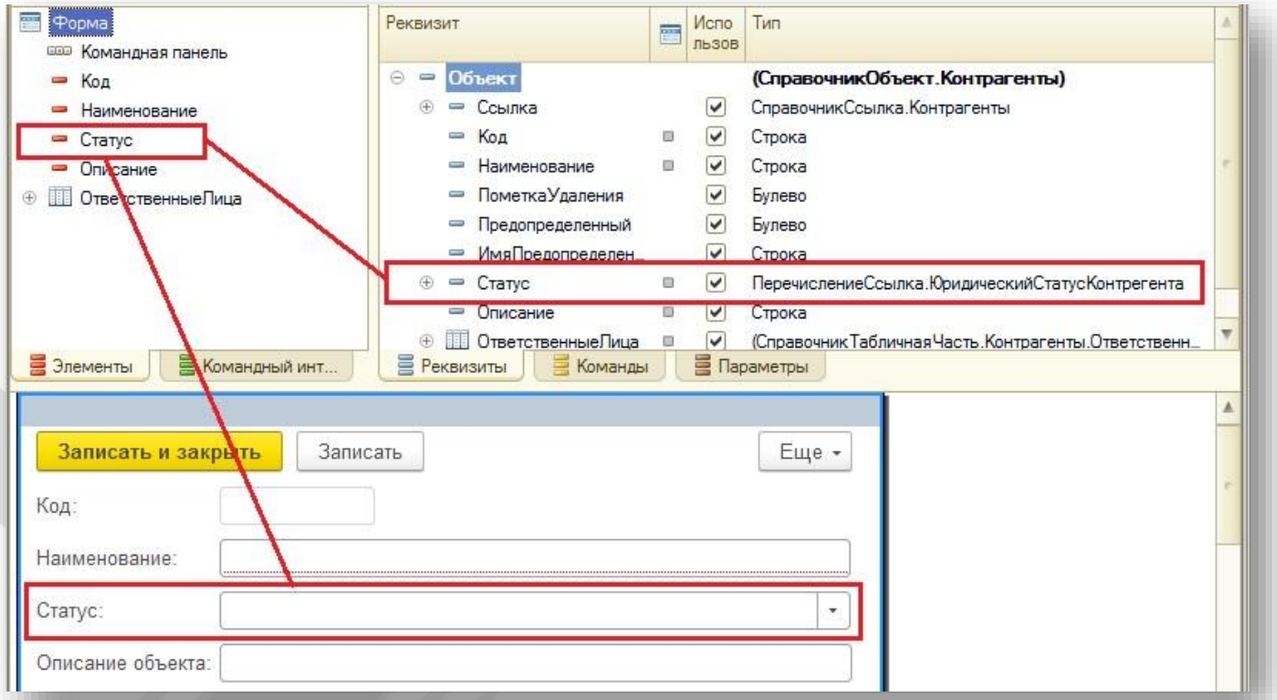
Перечисления. Значение перечисления.

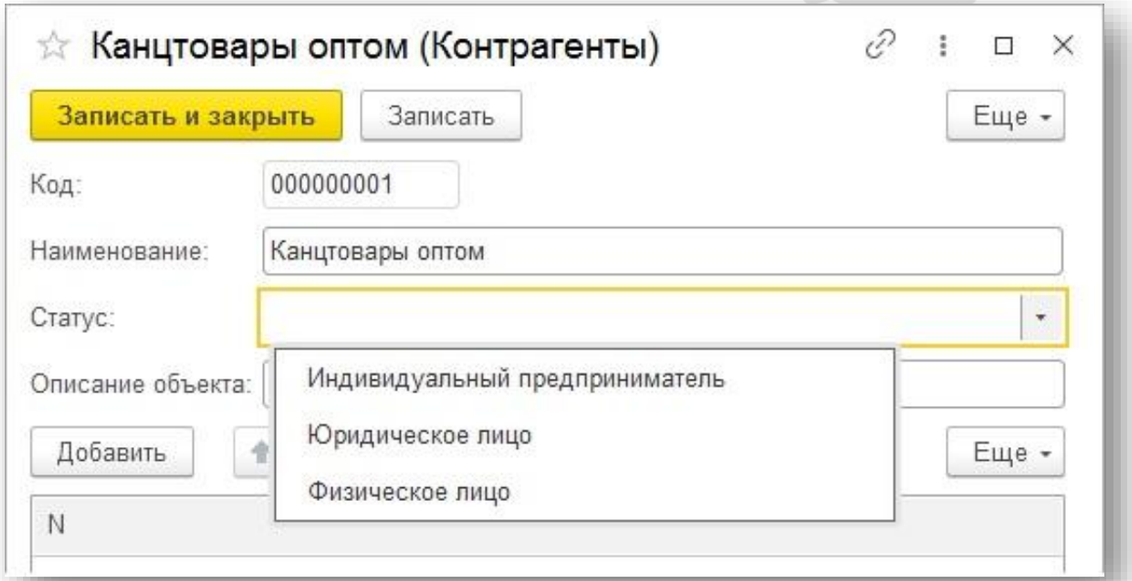
К имени значения перечисления происходит обращение при программной работе с перечислением, а синоним отображается пользователю в интерфейсе.

Точно также можно добавить несколько других значений.

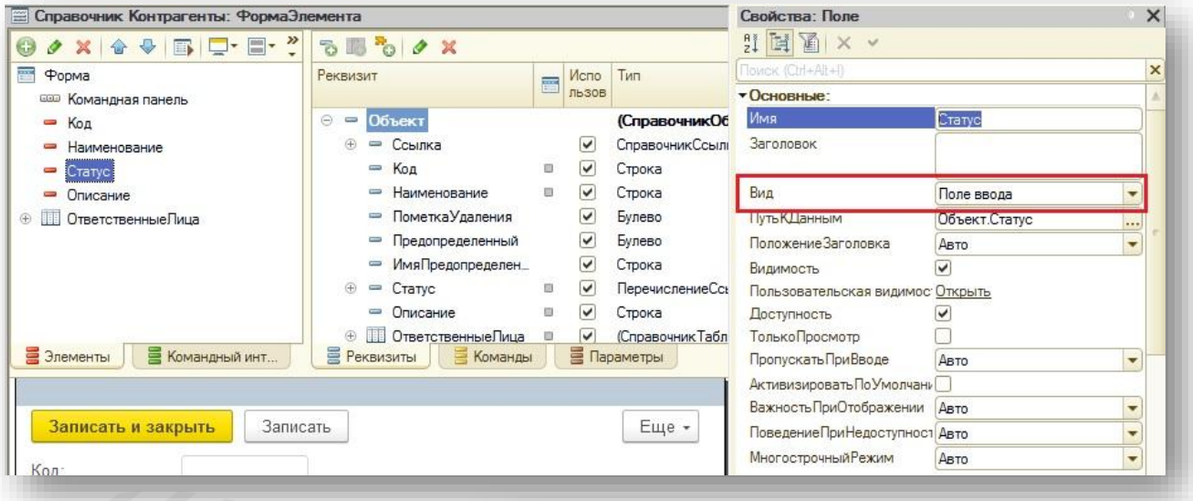


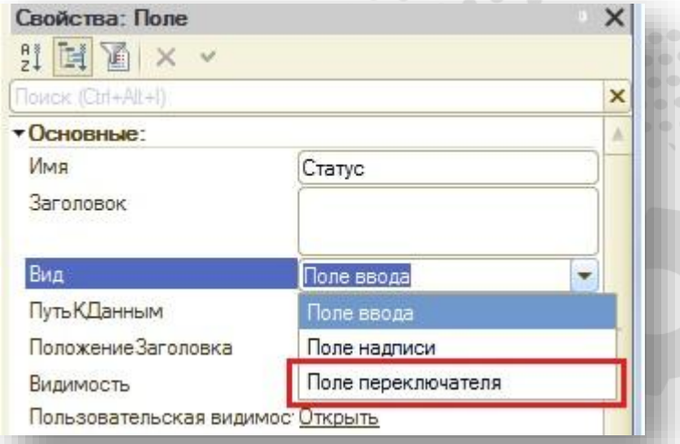
Перечисления. Перечисление на управляемой форме.

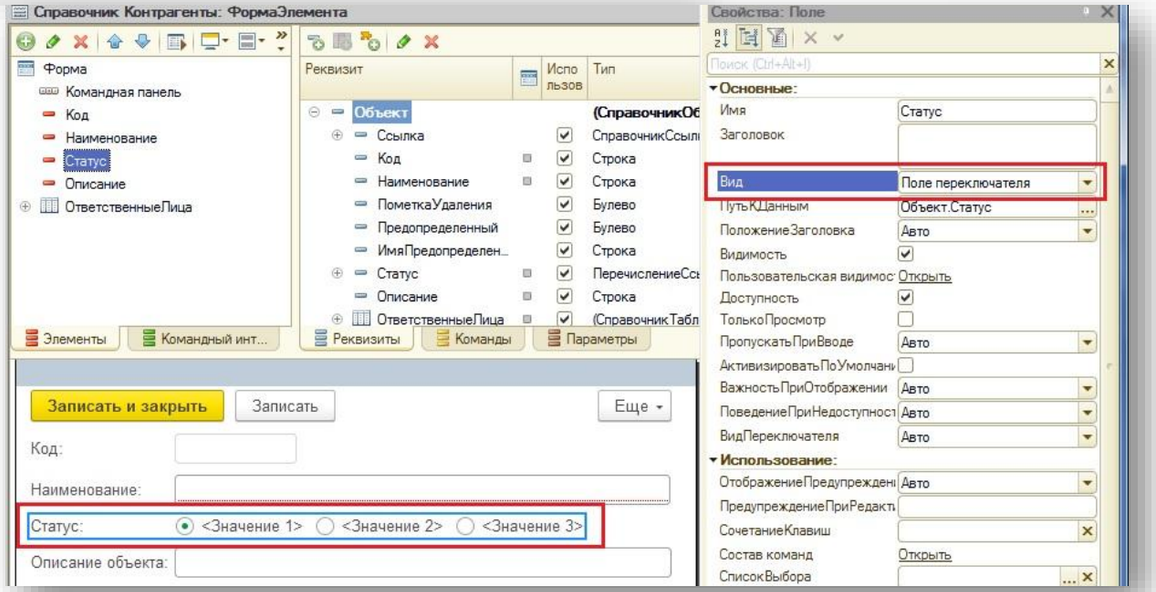




Перечисления. Перечисление на управляемой форме.

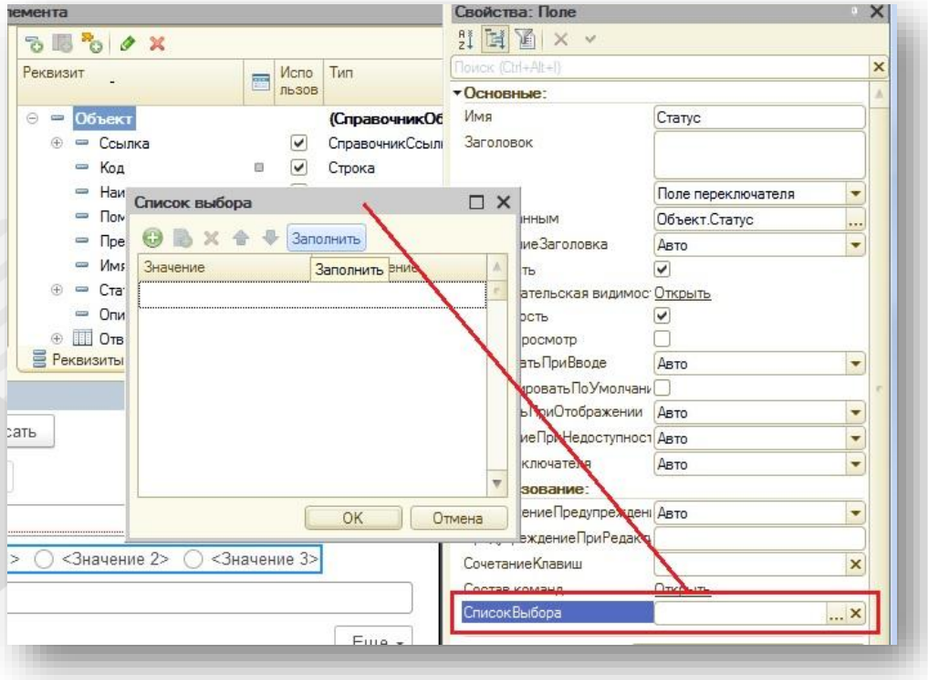


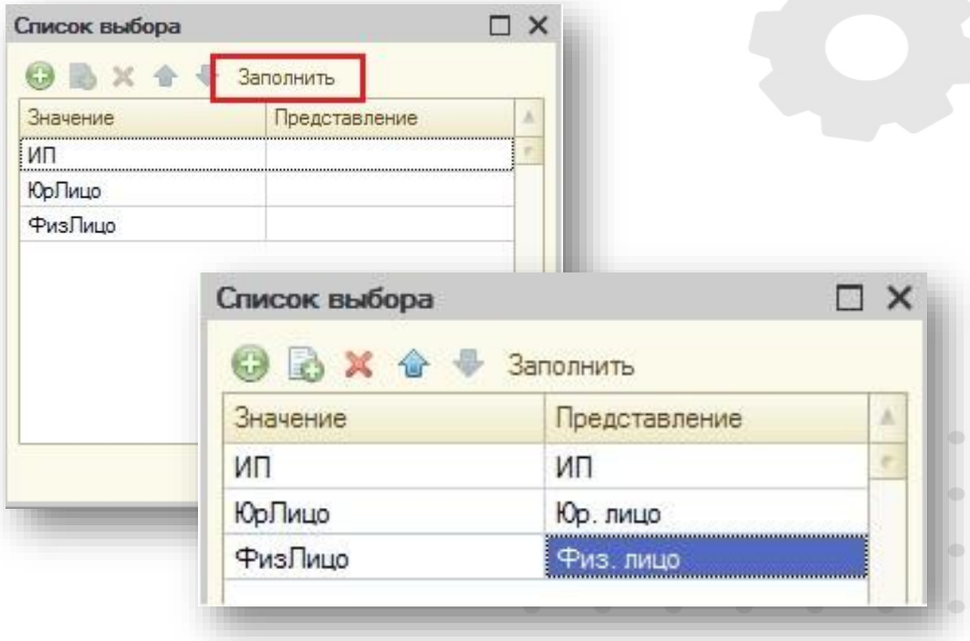




Перечисления. Перечисление на управляемой форме.

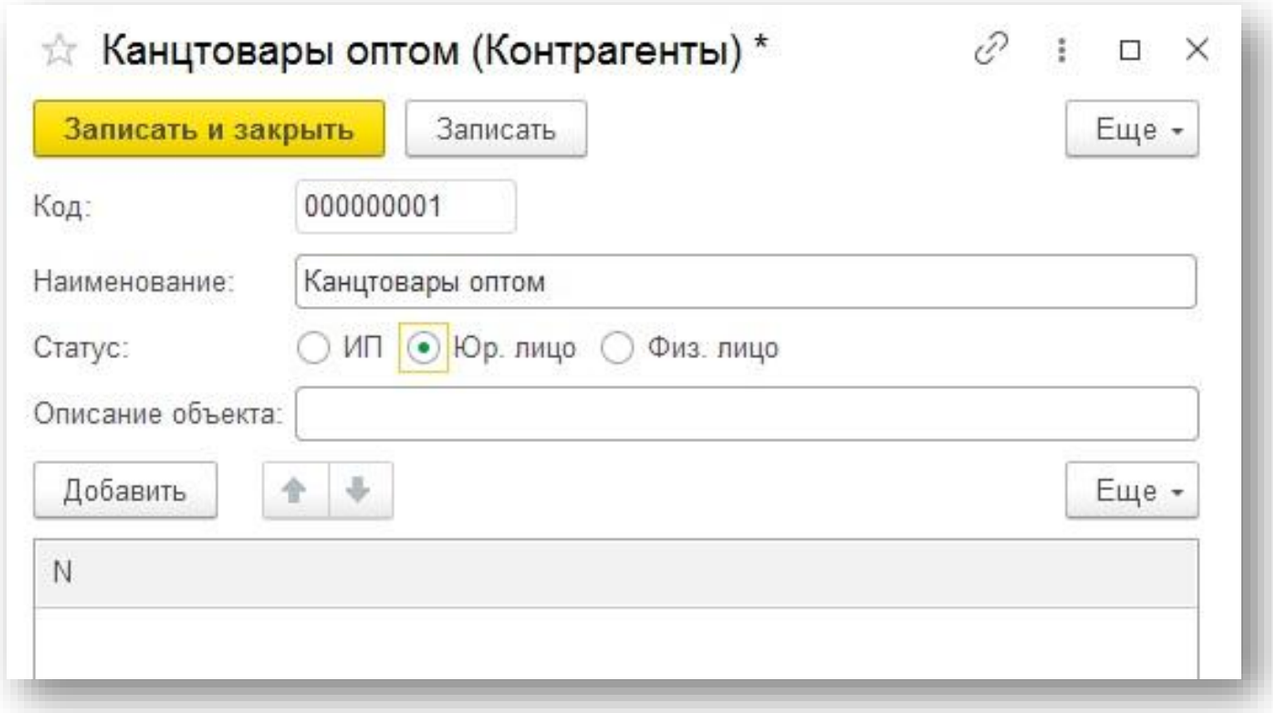
Можно автоматически заполнить список всеми значениями перечисления, нажав на кнопку «Заполнить» этой формы. Но представления значений, придется заполнить вручную.



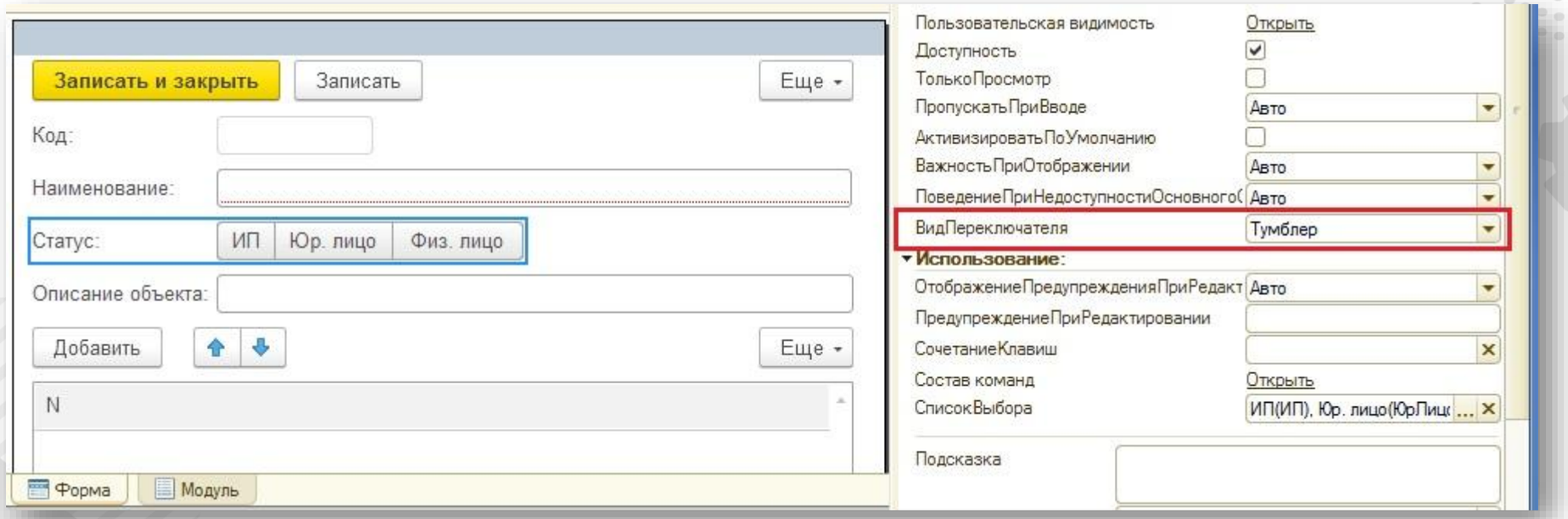


Перечисления. Перечисление на управляемой форме.

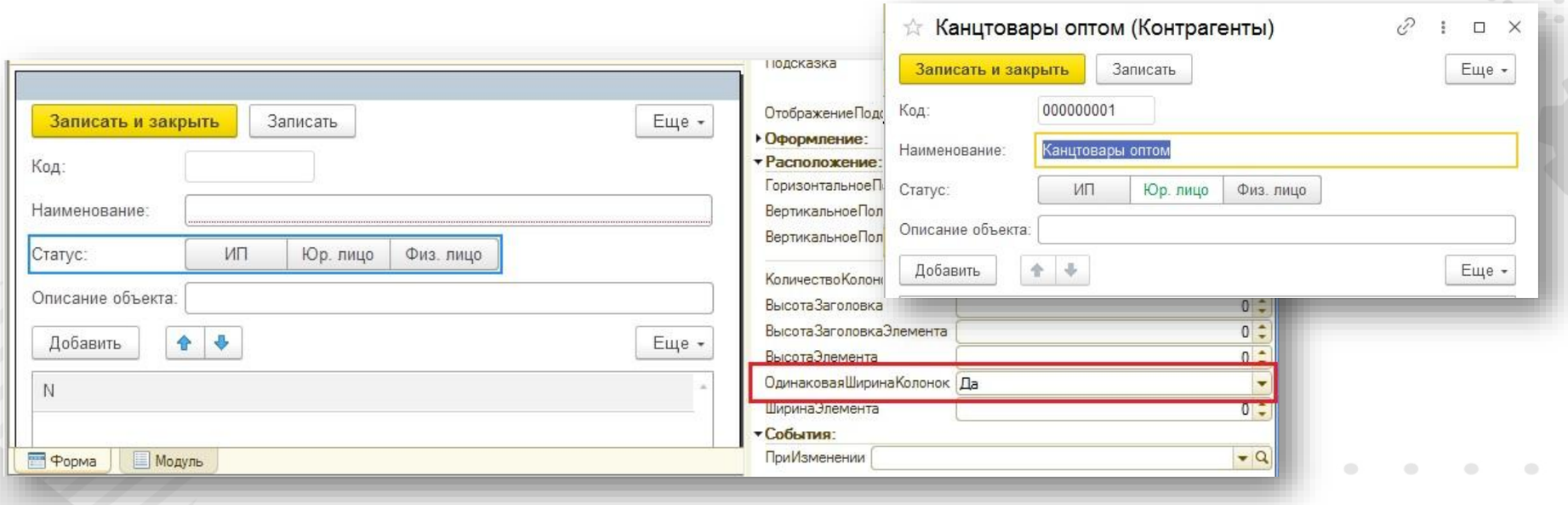
Также другой вид будет в пользовательском режиме «1С: Предприятия» .



Перечисления. Перечисление на управляемой форме.



Перечисления. Перечисление на управляемой форме.



Перечисления. Программная работа с перечислениями.

Программная работа с перечислениями в 1С отличается в зависимости от того в каком контексте мы работаем с этим перечислением: в серверном или клиентском (конкретно на тонком клиенте и веб-клиенте).

ПЕРЕЧИСЛЕНИЕ В СЕРВЕРНОМ КОНТЕКСТЕ

Для того, чтобы понять как работать со значениями перечисления в серверном контексте, рассмотрим простую задачу: будем программно создавать элемент справочника и присваивать ему конкретное значение перечисления.

Создадим программно нового контрагента, с уже конкретным значением перечисления. Делать это будем на управляемой форме. На этой форме создаём команду, серверный и клиентский обработчик команды.

Перечисления. Программная работа с перечислениями.

В серверном обработчике и будет создаваться новый элемент справочника.

```
&НаСервере
Процедура СоздатьСправочникНаСервере()
    НовыйКонтр = Справочники.Контрагенты.СоздатьЭлемент();
    НовыйКонтр.Наименование = "Иванов и Партнеры";
    НовыйКонтр.Статус = Перечисления.ЮридическийСтатусКонтрегента.ЮрЛицо;
    НовыйКонтр.Записать();
КонецПроцедуры

&НаКлиенте
Процедура СоздатьСправочник(Команда)
    СоздатьСправочникНаСервере();
КонецПроцедуры
```

В этом коде мы создаем объект справочника, заполняем его реквизиты какими-то значениями, а потом сохраняем. Реквизиту *Статус* присваивается конкретное значение перечисления *ЮридическийСтатусКонтреганта*.

Перечисления. Программная работа с перечислениями.

В серверном обработчике и будет создаваться новый элемент справочника.

```
&НаСервере
Процедура СоздатьСправочникНаСервере()
    НовыйКонтр = Справочники.Контрагенты.СоздатьЭлемент();
    НовыйКонтр.Наименование = "Иванов и Партнеры";
    НовыйКонтр.Статус = Перечисления.ЮридическийСтатусКонтрегента.ФизЛицо;
    НовыйКонтр.Записать();
КонецПроцедуры

&НаКлиенте
```

Табло - 1

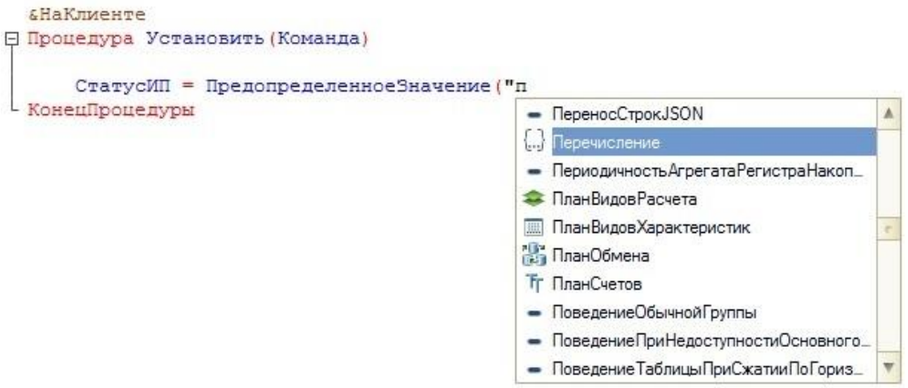
Выражение	Значение
Перечисления.ЮридическийСтатусКонтрегента	ПеречислениеМенеджер.ЮридическийСтатусКонтрегента
ИП	Индивидуальный предприниматель
ФизЛицо	Физическое лицо
ЮрЛицо	Юридическое лицо

Перечисления. Программная работа с перечислениями.

Чтобы получить в режиме тонкого клиента значение перечисления, нужно использовать метод глобального контекста ПредопределенноеЗначение. В этом случае код будет следующим.

```
&НаКлиенте
Процедура Установить(Команда)
    СтатусИП = ПредопределенноеЗначение("Перечисление.ЮридическийСтатусКонтрегента.ИП");
КонецПроцедуры
```

В параметре этого метода в виде строки необходимо написать путь к значению перечисления. Заметим, что в платформе 1С, после ввода круглой скобки и первой кавычки выпадают подсказки, и по этим подсказками можно сгенерировать нужную строку.

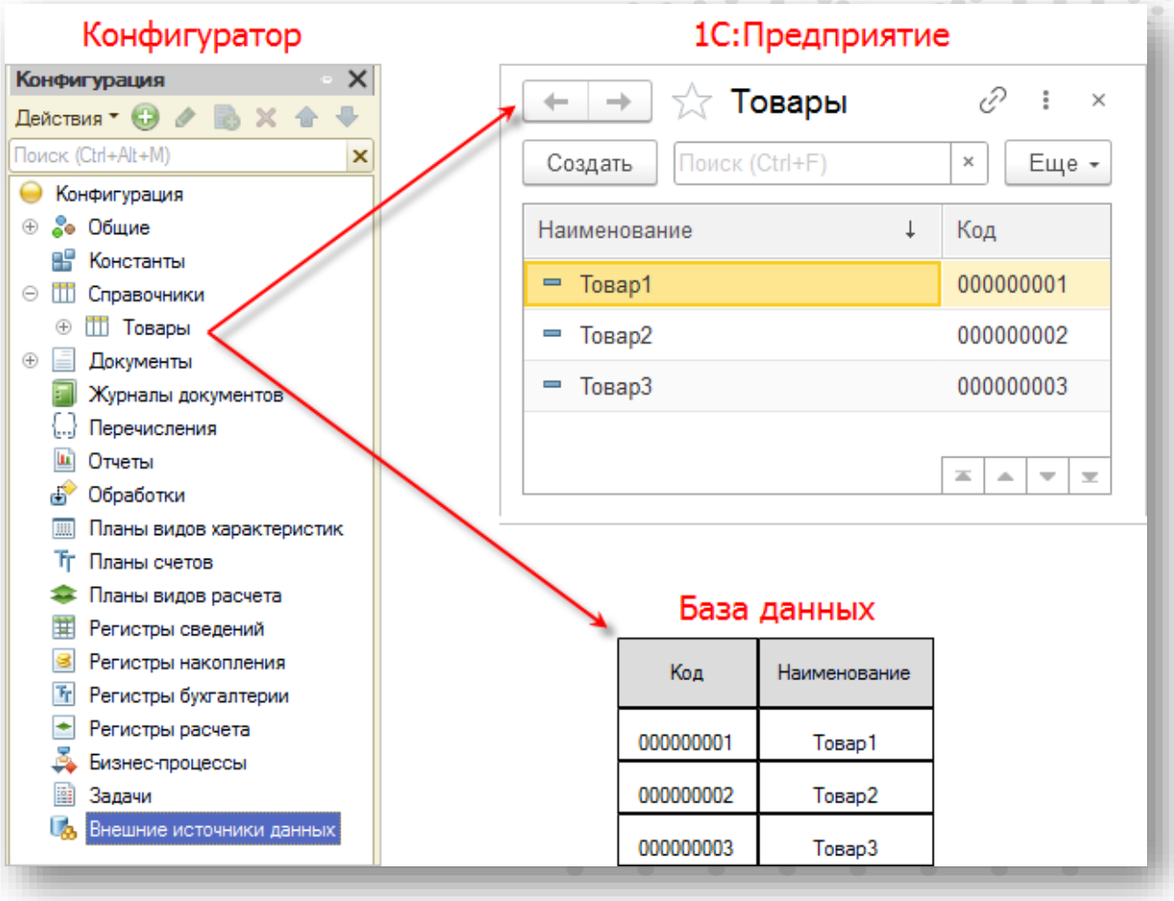


Справочники. Общие сведения

Справочники – это **прикладные объекты конфигурации**. Они позволяют хранить в информационной базе данные, **имеющие одинаковую структуру и списочный характер.**

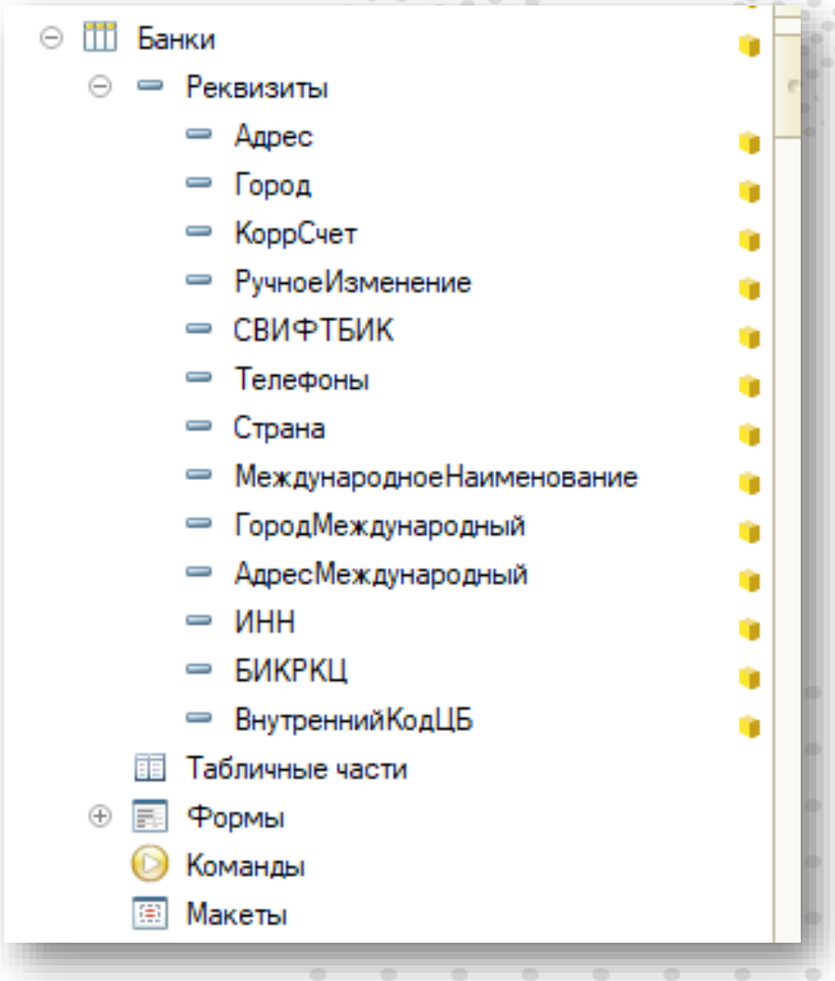
Как правило, в работе любой фирмы используются списки сотрудников, списки товаров, списки клиентов, поставщиков и т. д. Свойства и структура этих списков описываются в объектах конфигурации «*Справочник*», на основе которых платформа создает в базе данных таблицы для хранения информации из этих справочников

Справочник состоит из *элементов*. Например, для справочника сотрудников элементом является сотрудник, для справочника товаров – товар и т. д. **Пользователь в процессе работы может** самостоятельно добавлять новые элементы в справочник: например, добавить новых сотрудников, создать новый товар или внести нового клиента



Справочники. Структура. Реквизиты

Каждый элемент справочника, как правило, содержит некоторую дополнительную информацию, которая подробнее описывает этот элемент. Например, все элементы справочника *«Банки»* могут содержать дополнительную информацию об адресе, городе и др. Набор такой информации является одинаковым для всех элементов справочника, и для описания такого набора используются ***реквизиты*** объекта конфигурации *«Справочник»*, которые также, в свою очередь, являются объектами конфигурации

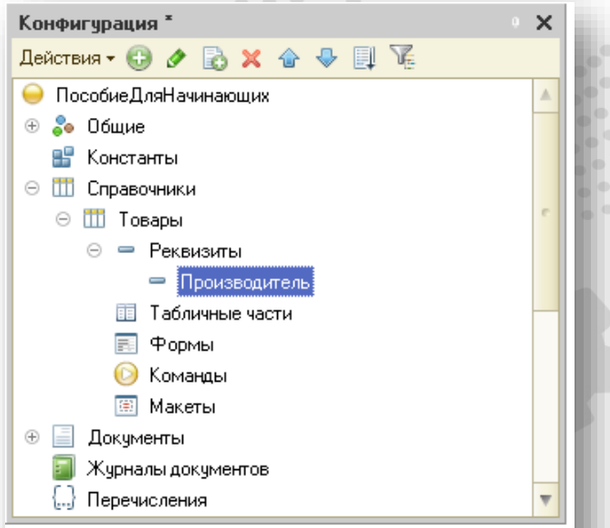


Справочники. Структура. Стандартные реквизиты

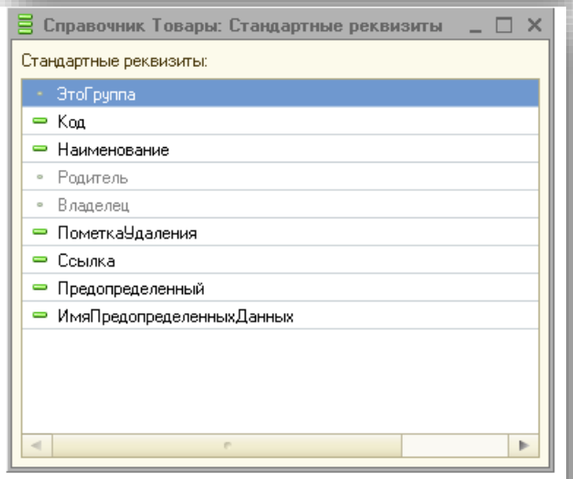
Поскольку эти объекты конфигурации логически связаны с объектом «*Справочник*», они называются *подчиненными* этому объекту

Большинство реквизитов разработчик создает самостоятельно, однако у каждого объекта конфигурации *Справочник* по умолчанию существует **набор стандартных реквизитов**: *Код* и *Наименование* и пр. Причем доступность стандартных реквизитов зависит от свойств справочника

Например, если справочник иерархический, у него будет доступен стандартный реквизит *Родитель*. Если справочник подчинен другому объекту конфигурации, у него будет доступен реквизит *Владелец*. Если установить длину стандартного реквизита *Код* равной нулю, то у справочника будет недоступен этот реквизит. То же самое относится к реквизиту *Наименование*. Однако как минимум либо *Код*, либо *Наименование* должны присутствовать в реквизитах справочника, иначе такой справочник не имеет смысла



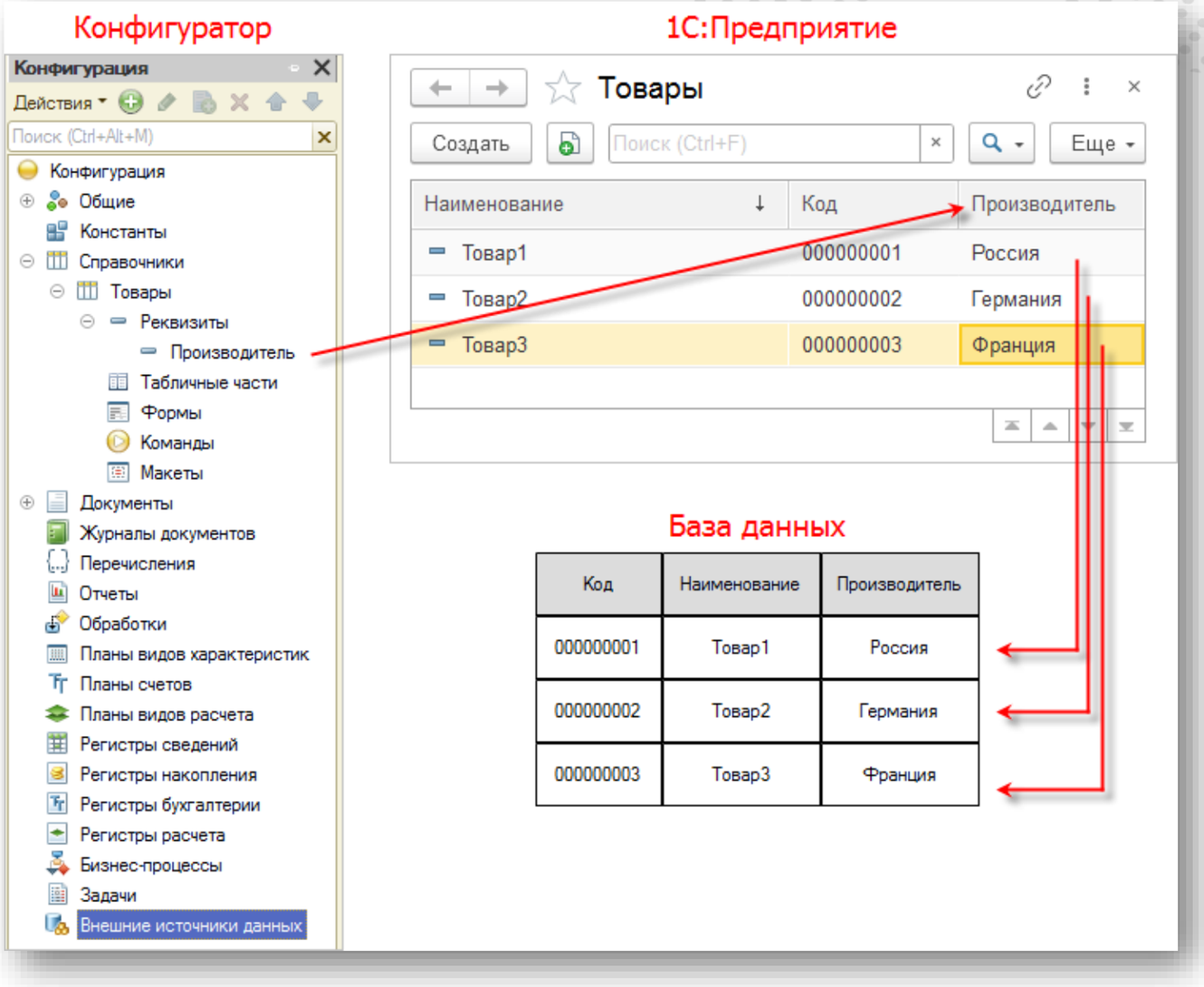
Реквизиты, созданные разработчиком



Стандартные реквизиты справочника

Справочники. Структура. Хранение в базе данных

В базе данных справочник хранится в виде таблицы, в строках которой расположены элементы списка, а каждому реквизиту (стандартному или созданному разработчиком) в этой таблице соответствует отдельный столбец. Соответственно, в ячейках этой таблицы хранится значение конкретного реквизита для конкретного элемента справочника

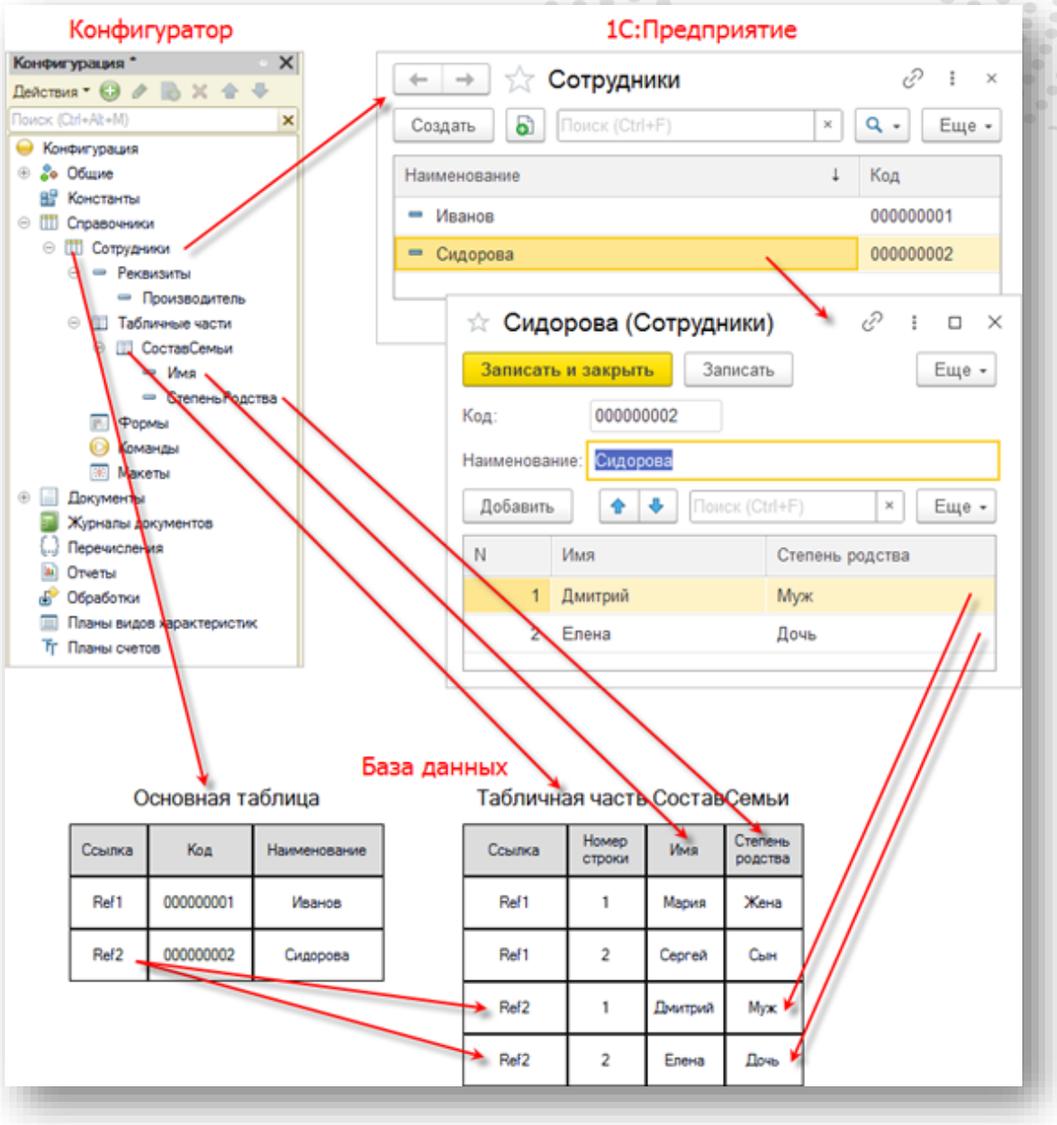


Справочники. Структура. Табличные части

Кроме этого, каждый элемент справочника может содержать некоторый набор информации, которая одинакова по своей структуре, но различна по количеству и предназначена для разных элементов справочника

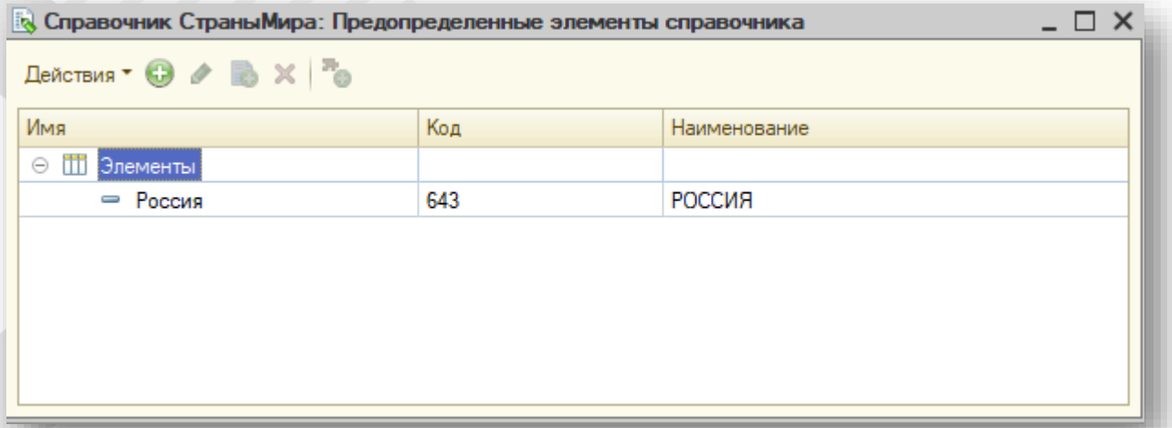
Так, например, каждый элемент справочника *Сотрудники* может содержать информацию о составе семьи сотрудника. Для одного сотрудника это будет только супруга, а у другого семья может состоять из супруги, сына и дочери

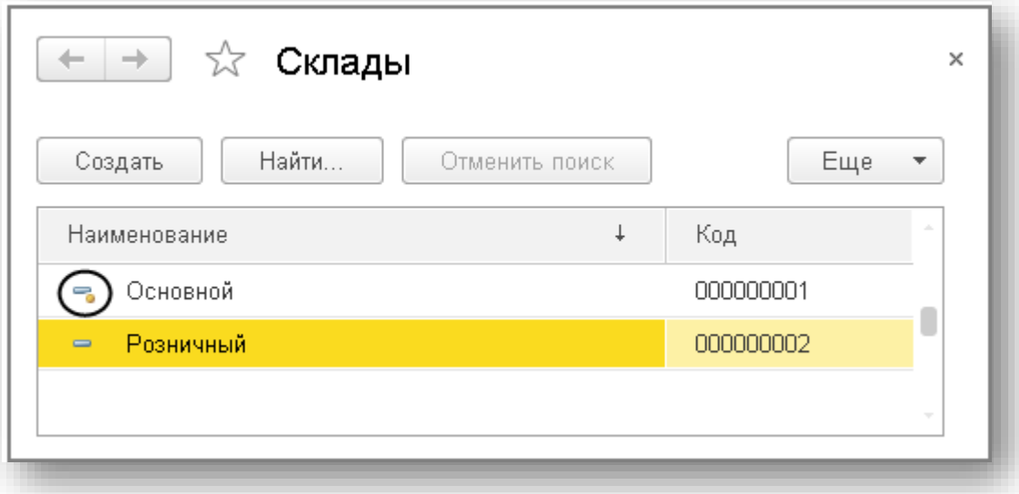
Для описания подобной информации могут быть использованы ***табличные части*** объекта конфигурации *Справочник*, являющиеся подчиненными ему объектами конфигурации. В этом случае в базе данных будут созданы дополнительные таблицы для хранения табличных частей, подчиненных конкретному элементу справочника



Справочники. Структура. Предопределенные элементы

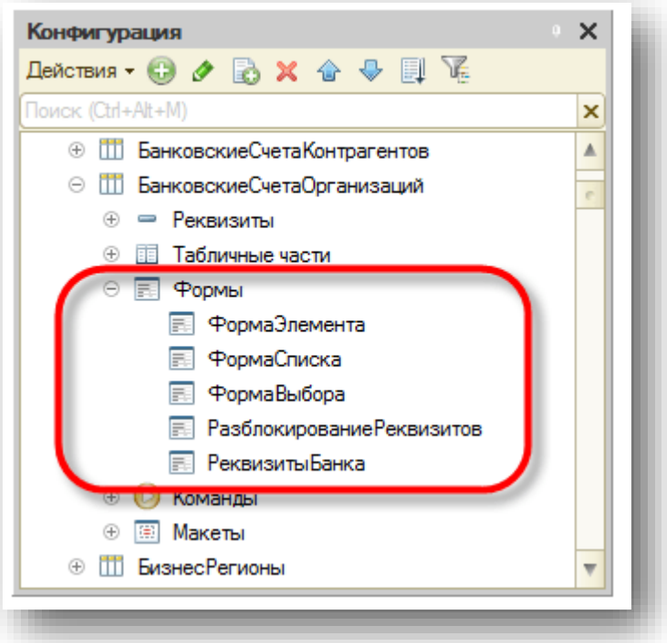
Порой возникают ситуации, когда необходимо, чтобы в справочнике некоторые элементы существовали всегда, независимо от действий пользователя. Допустим, логика бизнес-процессов на предприятии такова, что все товары сначала поступают на основной склад, а затем по мере надобности перемещаются на другие склады. В этом случае в справочнике *Склады* всегда должен существовать склад *Основной*, иначе приходование товаров будет выполнено неправильно. Объект конфигурации *Справочник* позволяет описать любое количество таких элементов справочника. Они называются ***предопределенными*** элементами справочника





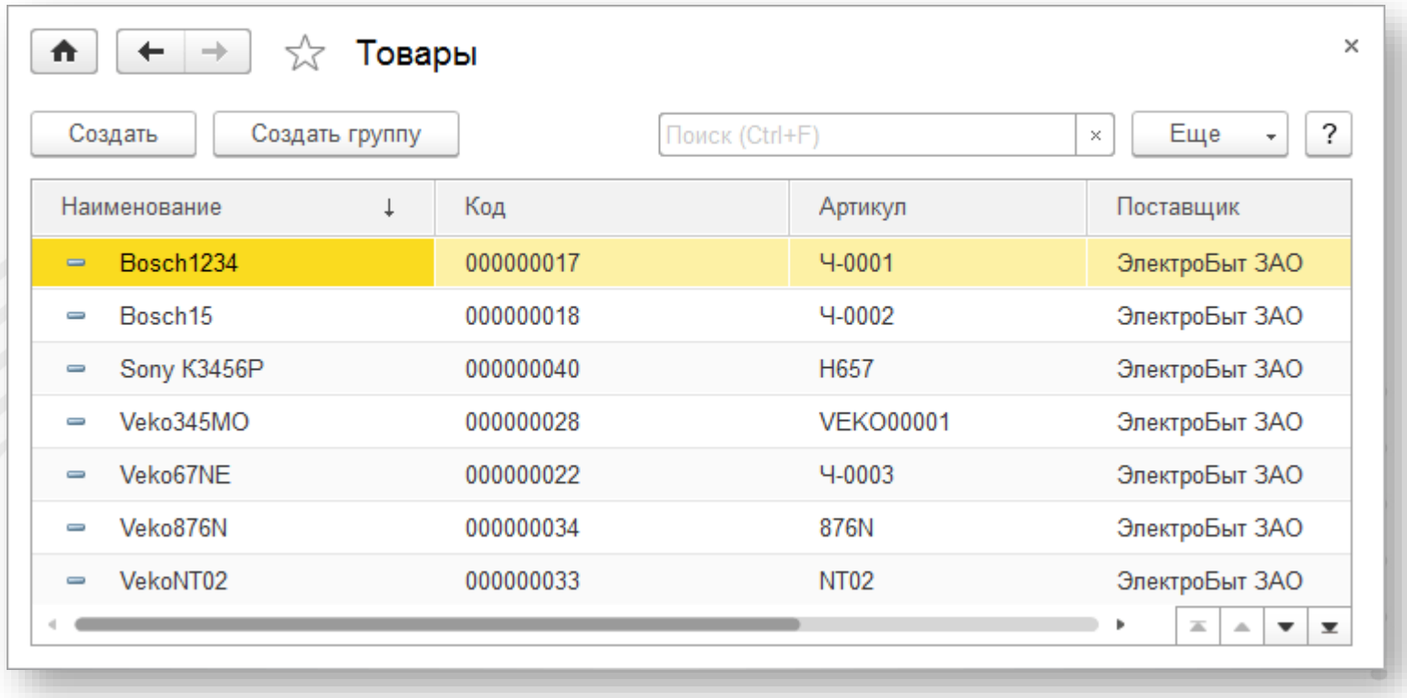
Справочники. Формы. Виды форм.

Для того чтобы пользователь мог просматривать и изменять данные, содержащиеся в справочнике, система поддерживает несколько **форм представления справочника**. Система может автоматически генерировать все нужные формы справочника. Наряду с этим разработчик имеет возможность создать собственные формы, которые система будет использовать вместо форм по умолчанию:



Справочники. Формы. Форма списка.

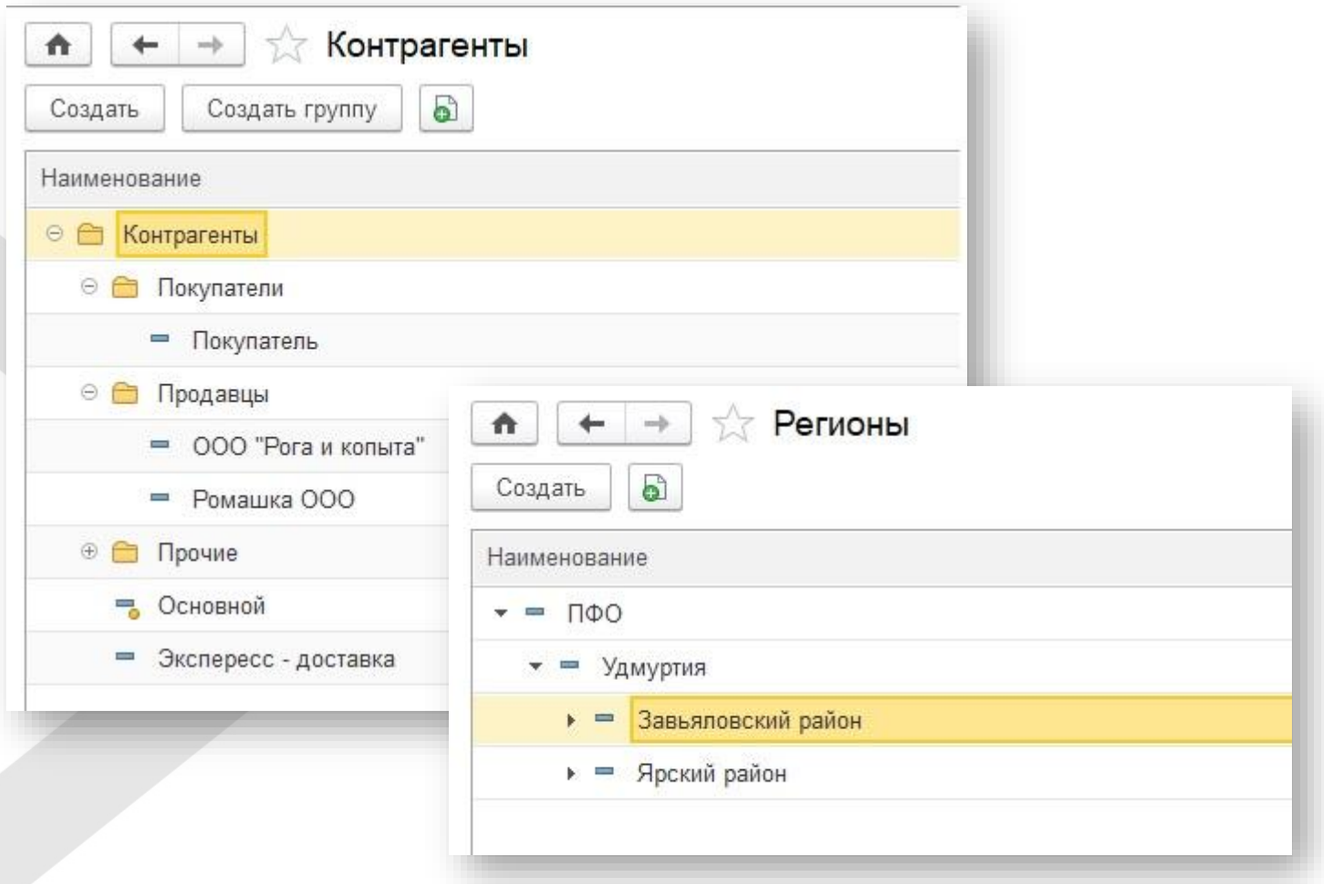
Для просмотра данных, содержащихся в справочнике, используется **форма списка**. Она позволяет выполнять навигацию по справочнику, добавлять, помечать на удаление и удалять элементы и группы справочника, перемещать элементы и группы. А также выполнять сортировку и отбор отображаемой информации по нескольким критериям:



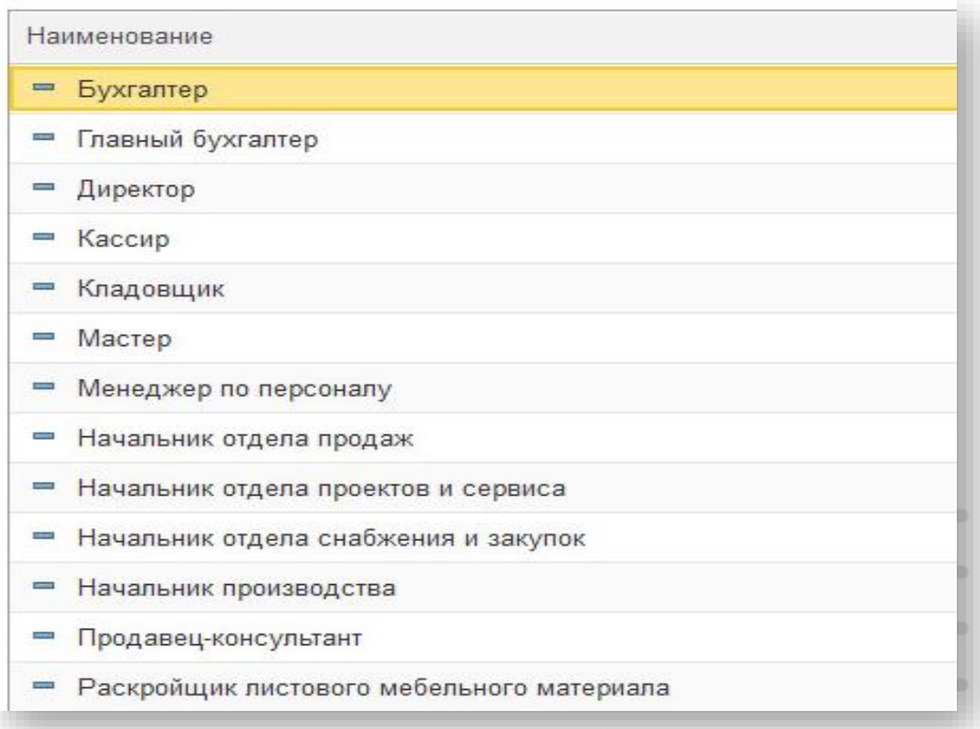
Справочники. Формы. Форма списка.

Форма списка может **представлять данные** в **иерархическом** и **не иерархическом виде.**

Иерархические справочники

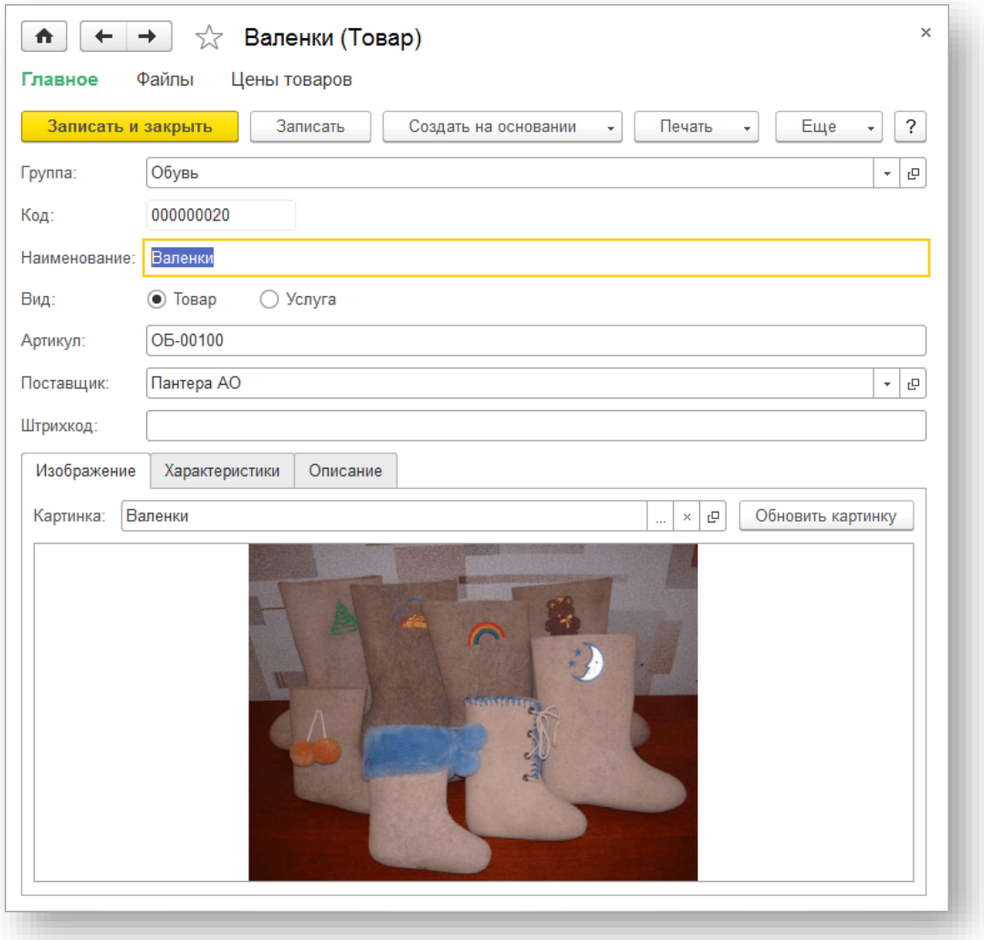


Не иерархический справочник



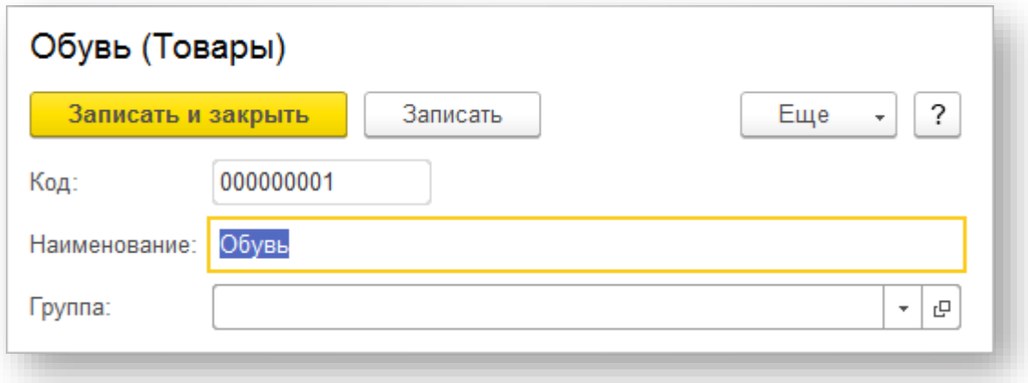
Справочники. Формы. Форма элемента.

Для просмотра и изменения данных отдельных элементов справочника используется **форма элемента**. Как правило, она представляет данные в удобном для восприятия и редактирования виде:



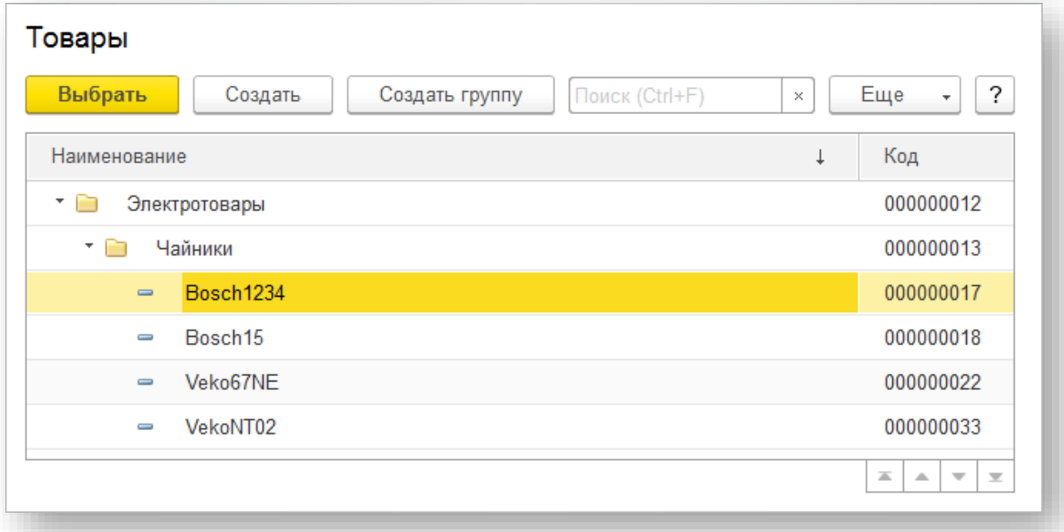
Справочники. Формы. Форма группы.

Система поддерживает отдельный **вид формы для групп справочника**. Это связано с тем, что, как правило, состав информации, относящейся к группе справочника, намного меньше, чем состав данных, имеющих отношение к отдельным элементам справочника:



Справочники. Формы. Форма выбора, форма выбора группы.

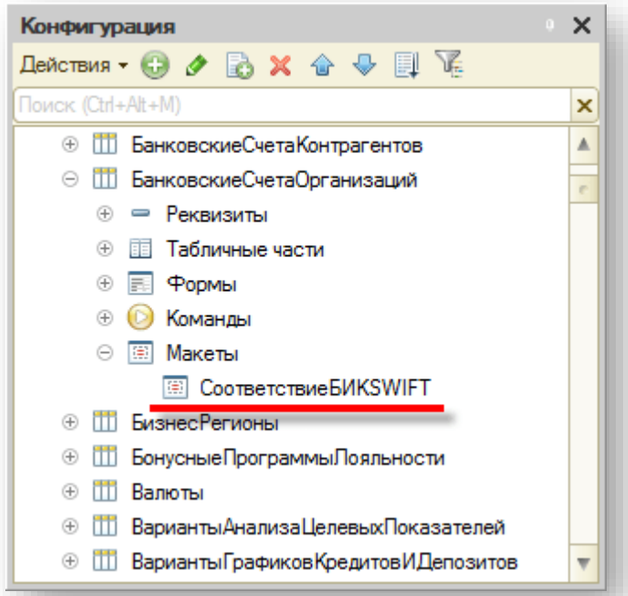
При различных действиях пользователя в прикладном решении возникает необходимость выбора конкретных элементов справочников. Это может потребоваться, например, при создании приходной накладной, в которой должны быть перечислены определенные товары. В этом случае система использует **форму выбора справочника**, которая, обычно, содержит минимальный набор информации, необходимой для выбора того, или иного элемента:



Кроме всех перечисленных форм, система поддерживает еще **форму выбора группы справочника**, предназначенную для выбора только среди групп, содержащихся в справочнике.

Справочники. Формы. Макеты.

Справочнику могут быть сопоставлены несколько макетов, содержащих данные, необходимые для обеспечения работы справочника:

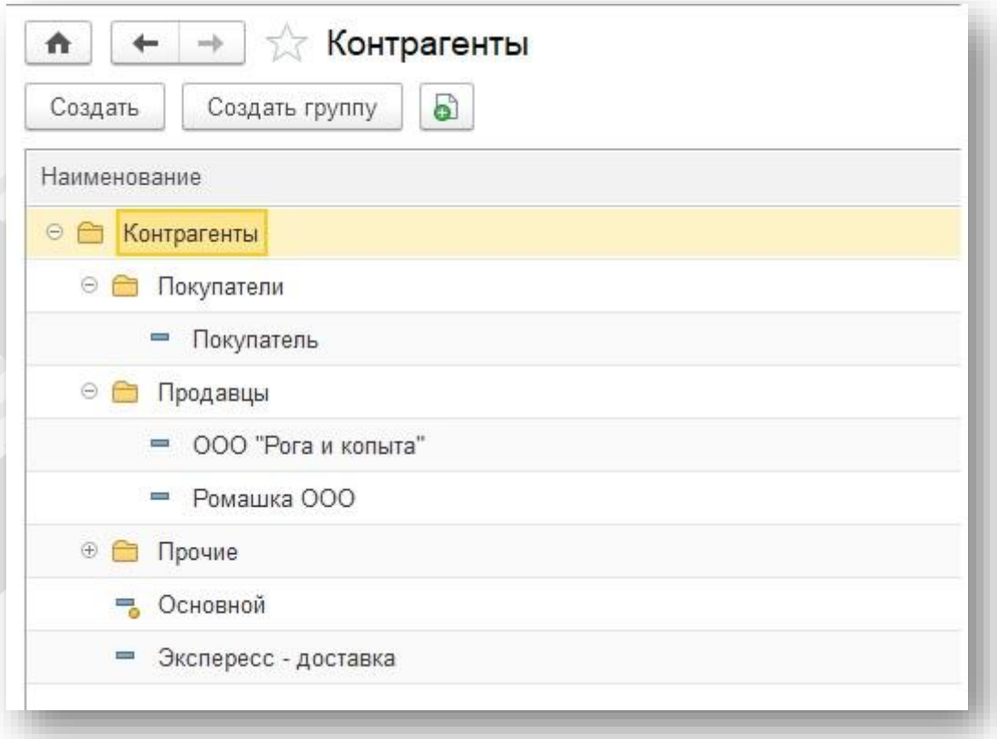


Макеты могут **использоваться** для формирования печатных форм справочника или для отображения дополнительной информации, имеющей отношение к справочнику.

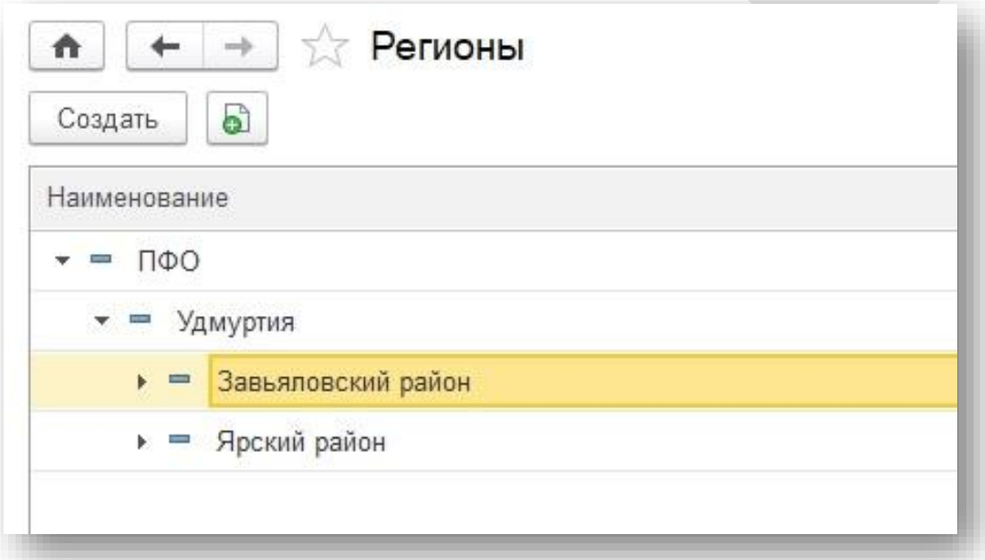
Справочники. Иерархический справочник.

Очень часто в справочнике необходимо настроить определенный порядок подчинения элементов. Для реализации подобных задач из простого справочника необходимо сделать **иерархический справочник**. В платформе 1С существует **два вида иерархии** – иерархия групп и элементов и иерархия элементов.

Список справочника с иерархией групп и элементов

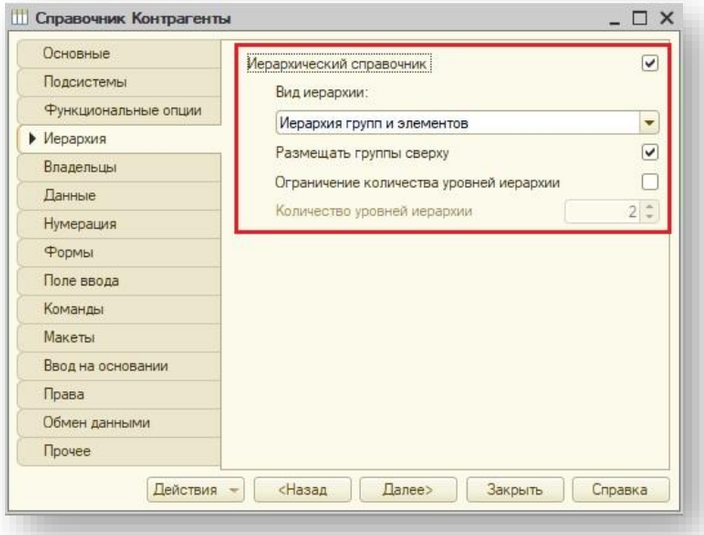


Список справочника с иерархией элементов



Справочники. Иерархический справочник. Иерархия групп и элементов.

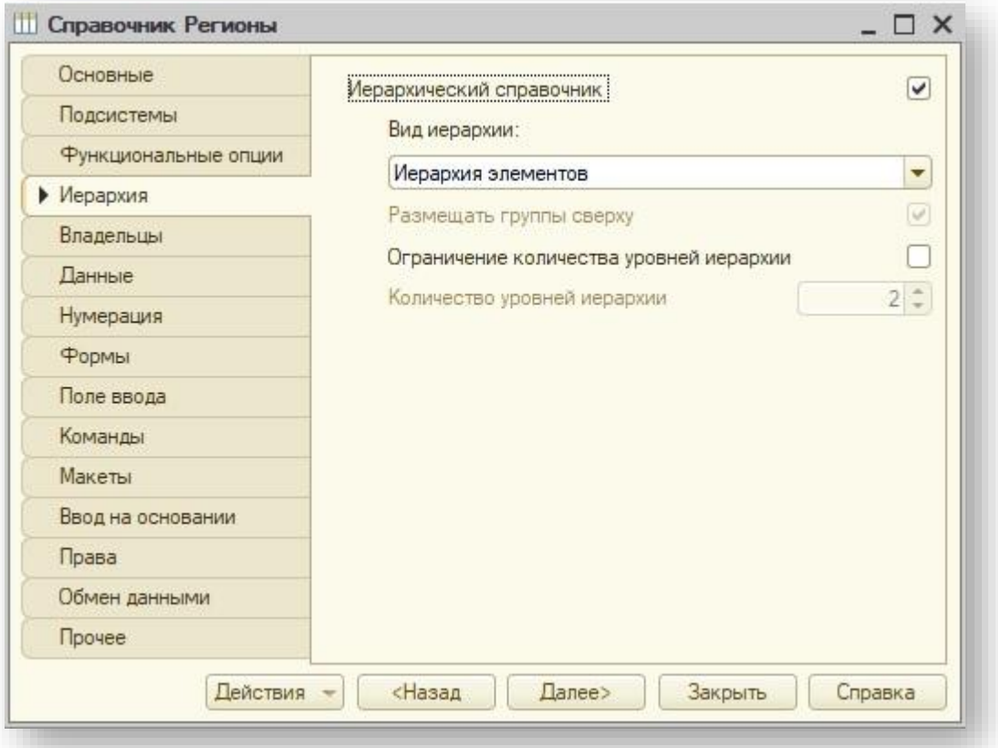
Для того, чтобы справочник **имел иерархию групп и элементов**, необходимо в редакторе справочника на закладке *Иерархия* установить **флаг** *Иерархический справочник*, а **свойству** *Вид иерархии* установить **значение** *Иерархия групп и элементов*. Тогда у справочника можно будет создавать группы, и размещать элементы в группах.



При иерархии групп и элементов существуют группы (каталоги), в которые входят или другие группы, или элементы. И группы, и элементы являются экземплярами объекта справочника.
Группа это тоже экземпляр объекта справочника, по которому не ведется какой-либо учёт (как правило), и который может быть родителем другого экземпляра, т.е. в группу могут входить другие группы и элементы справочников.

Справочники. Иерархический справочник. Иерархия элементов.

Иерархия элементов настраивается похожим способом: в редакторе справочника на **закладке** *Иерархия* устанавливается **флаг** *Иерархический справочник*, но **свойству** *Вид иерархии* следует установить **значение** *Иерархия элементов*.
После этого, каждый **элемент может быть** *родителем* другого элемента.

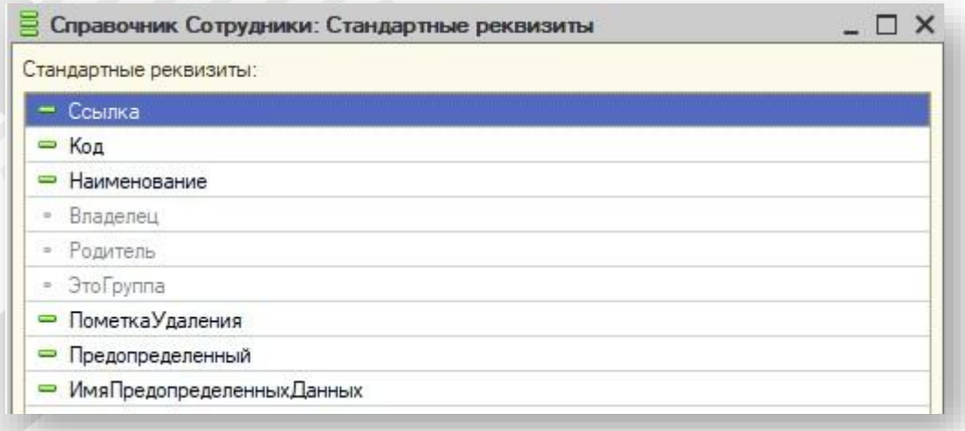


Справочники. Иерархический справочник. Родитель иерархического справочника.

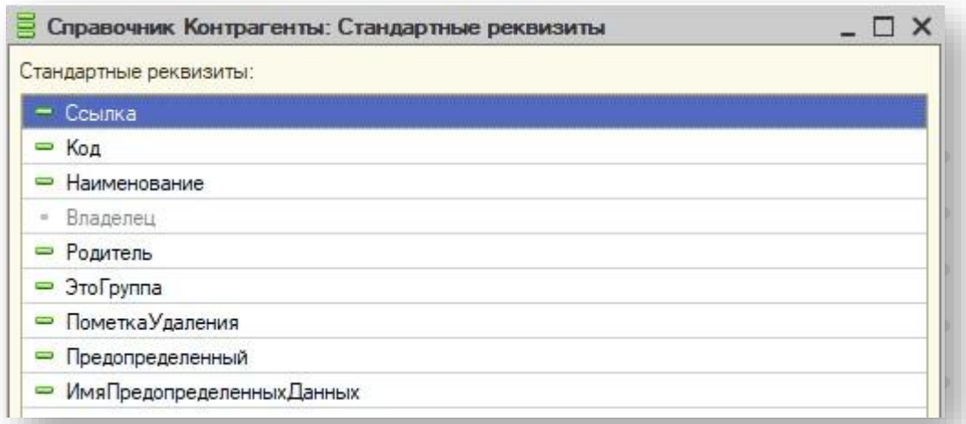
У **каждого элемента иерархического справочника** должен быть родитель. В случае ***иерархии групп и элементов*** **родитель** это **группа иерархического справочника**, в случае ***иерархии элементов***, **родитель** это **элемент**.

Родитель это **стандартный реквизит справочника**. Если иерархия отключена, то этот стандартный **реквизит не доступен**. А если иерархия включена, то этот **реквизит доступен**. Пример

Иерархия отключена, **реквизит не доступен**



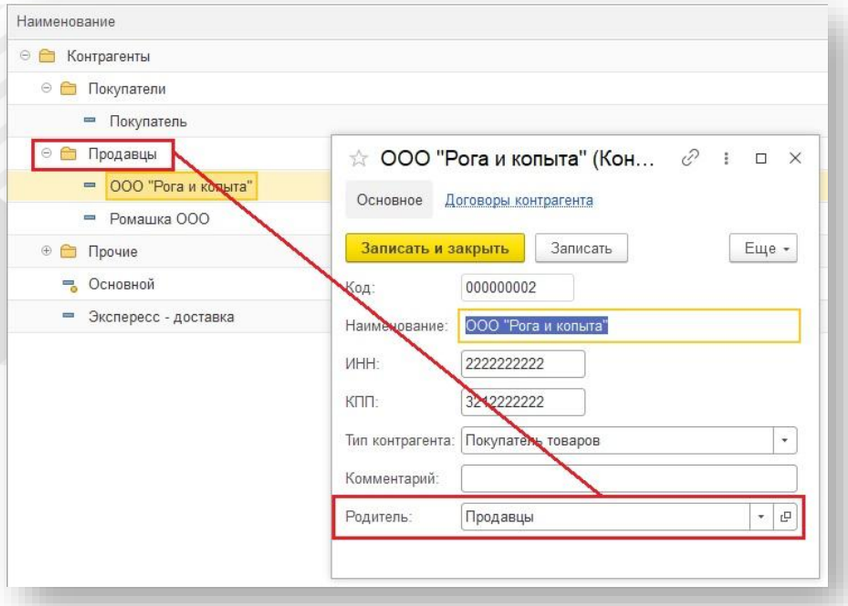
Иерархия включена, **реквизит доступен**



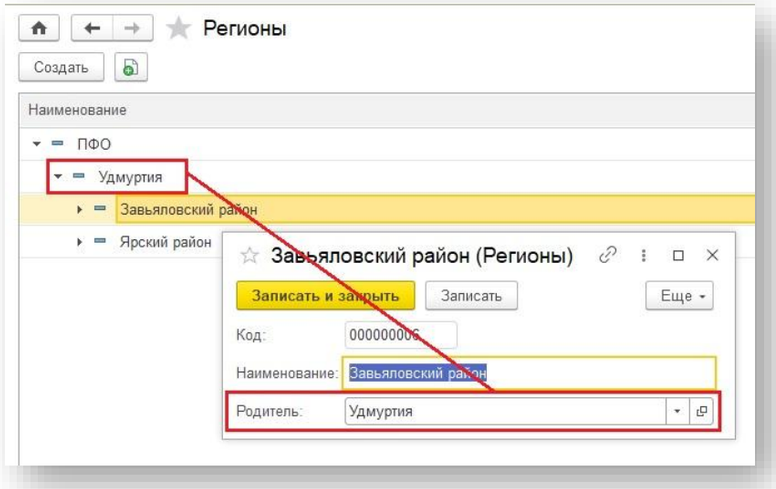
Справочники. Иерархический справочник. Форма элемента.

Если форма элемента справочника автоматически генерируется платформой (т.е. она не создана), то **реквизит** *Родитель* отображается на управляемой форме. И, **в случае иерархии групп и элементов** в нем указана группа, в которую входит этот элемент.
Если группа не указана, то значит элемент верхнего уровня.
Также, **родитель (элемент)** имеется у элементов справочника с иерархией элементов.

Форма элемента с иерархией групп и элементов



Форма элемента с иерархией элементов



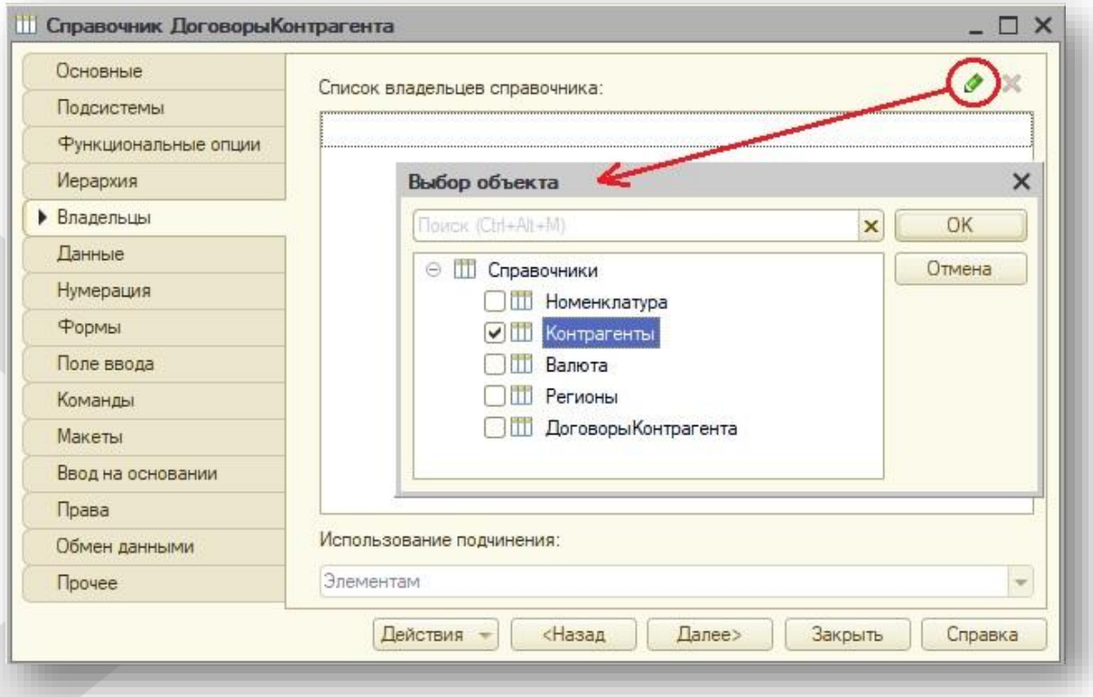
Справочники. Подчиненные справочники.

В прикладных задачах иногда возникают ситуации, когда элемент какого-нибудь справочника должен быть жестко подчинен элементу другого справочника. Например, у контрагентов могут существовать договоры, которые заключены с организацией, ведущей учёт. Тогда, экземпляр справочника *Договоры контрагента* должен быть подчинен справочнику *Контрагенты*, поскольку, мы не можем создать договор сам по себе, договор должен быть привязан к конкретному контрагенту.
Для реализации подобных задач **необходим подчиненный справочник**.

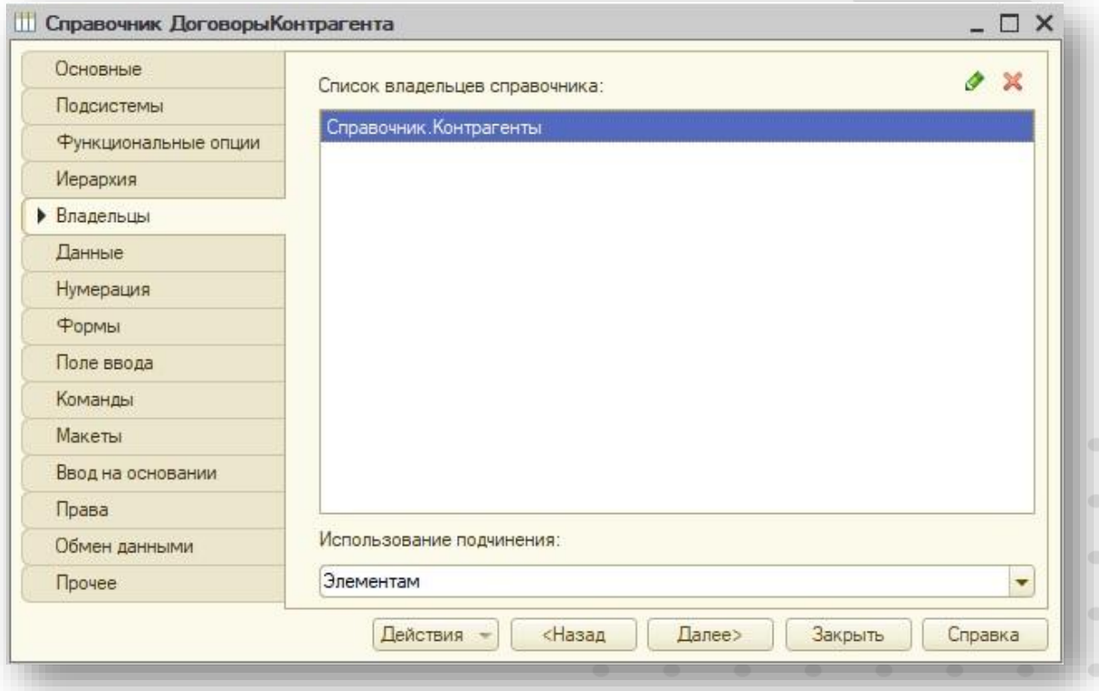
Справочники. Подчиненные справочники.

1) Для того, чтобы из справочника сделать подчиненный справочник, достаточно в редакторе справочника на **закладке** *Владельцы* указать владельца этого справочника.
2) После этого, справочник с заполненным списком на **закладке** *Владельцы* станет подчиненный, а справочник (или справочники) из списка на этой закладке будут *владельцами* данного справочника.

1)



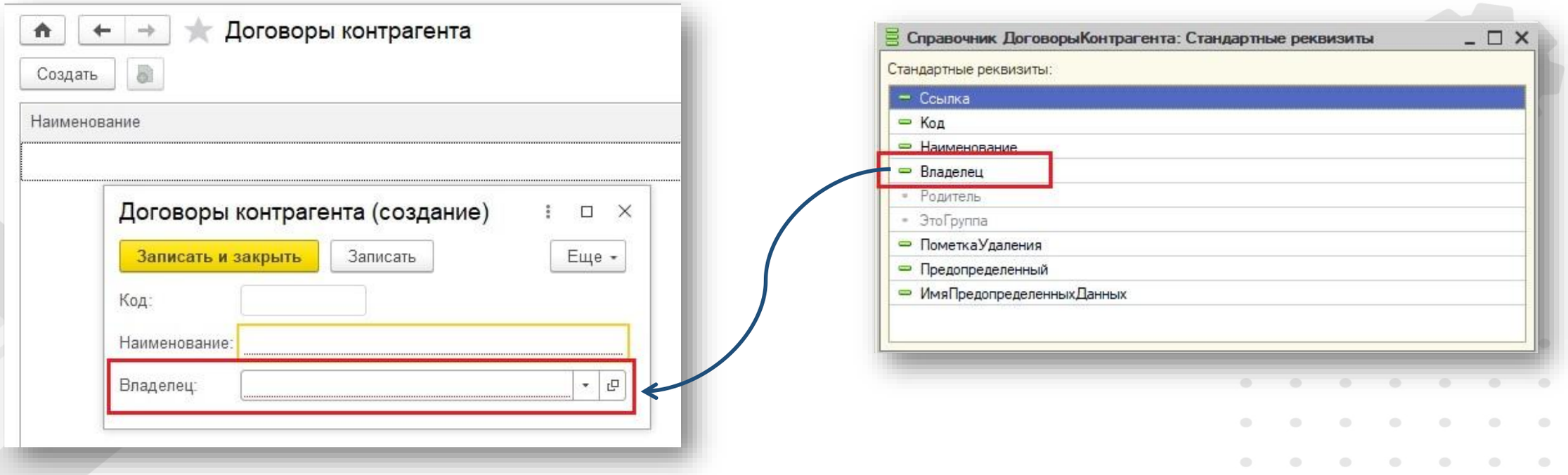
2)



После таких манипуляций, нельзя создать этот справочник (в рассматриваемом примере – *ДоговорыКонтрагента*) без указания владельца (*Контрагента*).

Справочники. Подчиненные справочники.

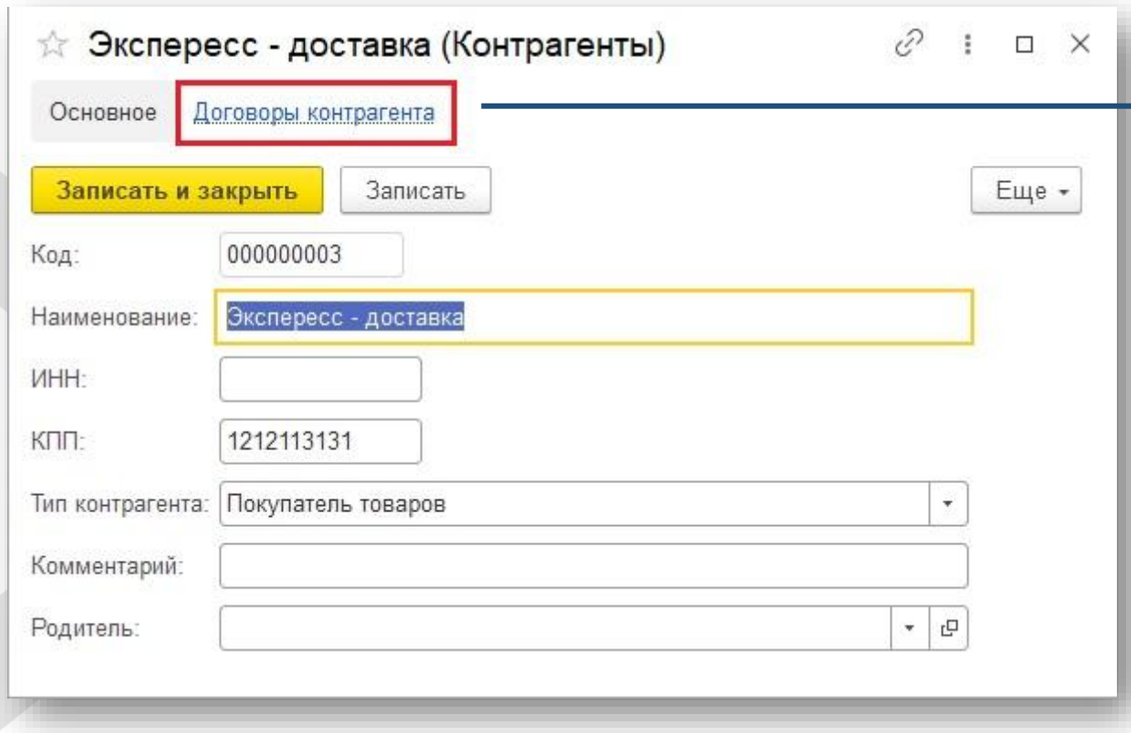
Если попробовать создать элемент справочника *ДоговорыКонтрагента*, то поле, в котором следует указать контрагента-владельца будет обязательным для заполнения. **Поле** называется *Владелец* и является стандартным реквизитом справочника.

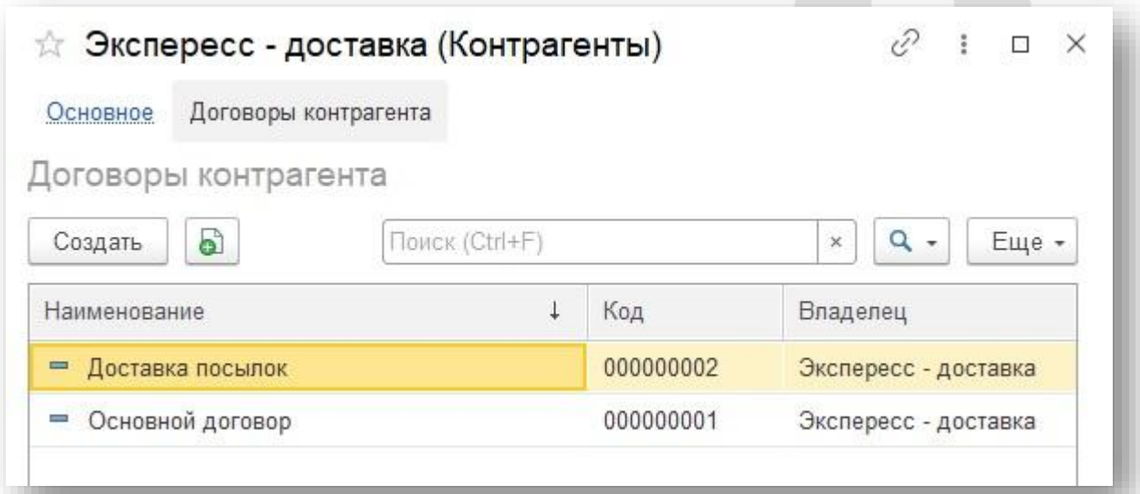


Этот **реквизит** становится активным, когда указывается владелец справочника на **закладке** *Владельцы*.

Справочники. Подчиненные справочники.

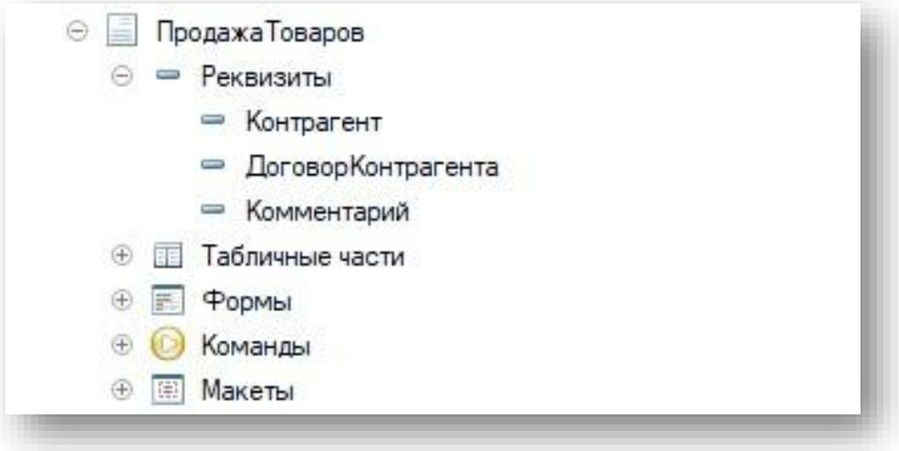
В тоже время, на **форме элемента справочника-владельца** появилась команда с названием подчиненного справочника. После выполнении этой команды, откроется форма списка с элементами подчиненного справочника, у которых владелец ссылка на открытый элемент справочника Контрагенты.





Справочники. Подчиненные справочники.

Часто возникают задачи, **когда на управляемой форме размещены поля ввода и для справочника владельца**, **и для подчиненного справочника**. И в этом случае нужно реализовать, чтобы в поле ввода подчиненного справочника отображались только те элементы, владелец которых указан в поле ввода справочника владельца.
Например, у документа *Продажа товаров* существуют реквизиты *Контрагент* и *Договор контрагента* соответствующих типов.



Справочники. Программная работа со справочниками.

Программное создание элемента справочника
Для программного создания элементов есть метод СоздатьЭлемент. Для программного создания групп — метод *СоздатьГруппу*:

```
НовыйТовар = Справочники.Товары.СоздатьЭлемент();
НовыйТовар.Наименование = "Клавиатура";
//...заполнение прочих реквизитов
//запись в базу
НовыйТовар.Записать();

НоваяГруппаТоваров = Справочники.Товары.СоздатьГруппу();
НоваяГруппаТоваров.Наименование = "Комплектующие";
//...заполнение прочих реквизитов
//запись в базу
НоваяГруппаТоваров.Записать();
```

Справочники. Программная работа со справочниками.

Программное изменение элементов справочника

Для того чтобы изменить реквизиты уже существующего элемента справочника нужно через ссылку получить объект методом ПолучитьОбъект, через объект изменить значения реквизитов и записать измененный объект в базу данных:

```
//функция вернула какую-то ссылку на справочник Товары
СсылкаНаТовар = СсылкаНаТовар();

//получам объект из ссылки
ТоварОбъект = СсылкаНаТовар.ПолучитьОбъект();
//меняем наименование
ТоварОбъект.Наименование = ТоварОбъект.Наименование + " (изменение)";
//записываем
ТоварОбъект.Записать();
```

Справочники. Программная работа со справочниками.

Программное копирование элемента справочника

Методом Скопировать можно создать копию элемента справочника. Данный метод можно вызывать как у ссылки, так и у объекта:

```
//через ссылку
КопияИзСсылки = СсылкаНаТовар.Скопировать();
КопияИзСсылки.Записать();

//через объект
ТоварОбъект = СсылкаНаТовар.ПолучитьОбъект();
КопияИзОбъекта = ТоварОбъект.Скопировать();
КопияИзОбъекта.Записать();
```

Справочники. Программная работа со справочниками.

Программное удаление элемента справочника

Для удаления применяются два метода: УстановитьПометкуУдаления и Удалить. Метод УстановитьПометкуУдаления только установит пометку на удаление, сам элемент не будет удален из базы данных. Метод Удалить удалит элемент из базы данных без проверки ссылочной целостности. То есть, если на этот элемент были ссылки в других местах, то теперь вместо ссылки будет надпись «Объект не найден». Не рекомендуется пользоваться методом Удалить. Важно отметить, что вызов метода УстановитьПометкуУдаления вызывает запись объекта. Дополнительно записывать объект не нужно.

```
ТоварОбъект = СсылкаНаТовар.ПолучитьОбъект();
ТоварОбъект.УстановитьПометкуУдаления(Истина);
//также будут помечены на удаление все подчиненные элементы

//без пометки удаления подчиненных элементов:
ТоварОбъект.УстановитьПометкуУдаления(Истина, Ложь);

//снятие пометки на удаление
ТоварОбъект.УстановитьПометкуУдаления(Ложь);

//удаление без проверки ссылочной целостности
ТоварОбъект.Удалить();
//также удалит все вложенные элементы
```

Справочники. Программная работа со справочниками.

Программный поиск по справочнику
Для поиска используются методы *НайтиПоКоду*, *НайтиПоНаименованию* и *НайтиПоРеквизиту*.

Найти По Коду

```
//поиск по коду
ТоварСсылка = Справочники.Товары.НайтиПоКоду("000000008");

//по полному коду. Полный код включает в себя все коды родителей:
//код родителя/код элемента
ТоварСсылка = Справочники.Товары.НайтиПоКоду("000000003/000000008", Истина);

//поиск в пределах одной группы, Группа - ссылка на группу справочника
ТоварСсылка = Справочники.Товары.НайтиПоКоду("000000008",, Группа);

//поиск в пределах владельца, Владелец - ссылка на владельца
ТоварСсылка = Справочники.Товары.НайтиПоКоду("000000008",,, Владелец);
```

Справочники. Программная работа со справочниками.

Найти По Наименованию

```
//поиск по наименованию, должна совпадать левая часть
ТоварСсылка = Справочники.Товары.НайтиПоНаименованию("Смартфон");
//будут найдены и "Смартфон" и "Смартфон самсунг", но метод вернет первый
//попавшийся элемент

//точное совпадение, кроме хвостовых пробелов
//будет найден только Смартфон
ТоварСсылка = Справочники.Товары.НайтиПоНаименованию("Смартфон", Истина);

//поиск в пределах одной группы, Группа - ссылка на группу справочника
ТоварСсылка = Справочники.Товары.НайтиПоНаименованию("Смартфон",, Группа);

//поиск в пределах владельца, Владелец - ссылка на владельца
ТоварСсылка = Справочники.Товары.НайтиПоНаименованию("Смартфон",,,Владелец);
```

Для обоих методов если элемент не будет найден, то метод вернет пустую ссылку. Если для справочника указана нулевая длина кода или наименования, то метод вернет Неопределено.

Справочники. Программная работа со справочниками.

Найти По Реквизиту

```
//поиск по реквизиту, 1-ый параметр имя реквизита, 2-ой значение
ТоварСсылка = Справочники.Товары.НайтиПоРеквизиту("Артикул", "Арт001");

//поиск в пределах одной группы, Группа - ссылка на группу справочника
ТоварСсылка = Справочники.Товары.НайтиПоРеквизиту("Артикул", "Арт001", Группа);

//поиск в пределах владельца, Владелец - ссылка на владельца
ТоварСсылка = Справочники.Товары.НайтиПоРеквизиту("Артикул","Арт1",,Владелец);
```

Если элемент не будет найден, то метод вернет пустую ссылку. Если есть несколько элементов с таким значением реквизита, то вернется первый попавшийся элемент.

Справочники. Программная работа со справочниками.

Выборка справочника

Для выборки всех элементов справочника используются методы Выбрать и ВыбратьИерархически. Метод Выбрать получает прямую выборку. Метод ВыбратьИерархически получает выборку с учетом иерархии, сразу после выборки группы будут идти элементы из этой группы. В обоих методах можно выполнять отбор по родителю, по владельцу, а также по индексированным полям. Последним параметром можно указать поля для сортировки выборки:

```
//отбор по артикулу. Поле артикул должно быть проиндексировано
//в отборе может быть только одно поле.
ОтборАрт = Новый Структура("Артикул", "");
ВыборкаСпр = Справочники.Товары.Выбрать(,, ОтборАрт, "Наименование Убыв");
Пока ВыборкаСпр.Следующий() Цикл
        Сообщить(ВыборкаСпр.Наименование);
КонецЦикла;
//выборка будет отсортирована по убыванию поля Наименование по умолчанию сортируется по основному представлению. Можно указать только код, наименование или проиндексированный реквизит примитивного типа.

//отбор по родителю или владельцу:
ВыборкаСпр = Справочники.Товары.Выбрать(Группа);
ВыборкаСпр = Справочники.Товары.Выбрать(, Владелец);
```

```
//иерархическая выборка, сначала группы, потом элементы из этой группы
ВыборкаСпр = Справочники.Товары.ВыбратьИерархически();
//можно получить уровень элемента (для верхнего = 0)
Сообщить(ВыборкаСпр.УровеньВВыборке());

//из выборки можно получить объект
Пока ВыборкаСпр.Следующий() Цикл
        ТоварОбъект = ВыборкаСпр.ПолучитьОбъект();
КонецЦикла;
```

Справочники. Программная работа со справочниками.

Прочие методы:

```
//получение макета
МакетСпр =
Справочники.Товары.ПолучитьМакет("ИмяМакета");
//можно через объект
МакетСпр = ТоварОбъект.ПолучитьМакет("ИмяМакета");

//получить полное наименование или код с учетом
вложенности через
ПолноеНаименование =
ТоварСсылка.ПолноеНаименование();
ПолныйКод = ТоварСсылка.ПолныйКод();

//принадлежит ли элемент определенной группе
Если
ТоварСсылка.ПринадлежитЭлементу(ССылкаНаГруппу)
Тогда
    Сообщить("Принадлежит");
КонецЕсли;
```

```
//уровень вложенности, если не в группе, то вернет 0
УровеньИерархии = ТоварСсылка.Уровень();

//определить пустая ссылка или нет
Если ТоварСсылка.Пустая() Тогда
    //пустая
КонецЕсли;

//получение пустой ссылки
ПустойТовар = Справочники.Товары.ПустаяСсылка();
```

Рекомендованная литература

- М.Г. Радченко, Е.Ю. Хрусталева, **1С:Предприятие 8.3. Практическое пособие разработчика**. Примеры и типовые приемы.